C++ Standard Library Ready Issues to be moved in Croydon, Mar. 2026

Doc. no. P4145R0

Date: 2026-03-25

Audience: WG21
Reply to: Jonathan Wakely <lwgchair@gmail.com>

Ready Issues

2414. Member function reentrancy should be implementation-defined

Section: 16.4.6.9 [reentrancy] **Status:** Ready **Submitter:** Stephan T. Lavavej **Opened:** 2014-07-01 **Last modified:** 2026-02-20

Priority: 3

View all other issues in [reentrancy].

Discussion:

N3936 16.4.6.9 [reentrancy]/1 talks about "functions", but that doesn't address the scenario of calling different member functions of a single object. Member functions often have to violate and then re-establish invariants. For example, vectors often have "holes" during insertion, and element constructors/destructors/etc. shouldn't be allowed to observe the vector while it's in this invariant-violating state. The [reentrancy] Standardese should be

extended to cover member functions, so that implementers can either say that member function reentrancy is universally prohibited, or selectively allowed for very specific scenarios.

(For clarity, this issue has been split off from LWG 2382[(i)].)

[2014-11-03 Urbana]

AJM confirmed with SG1 that they had no special concerns with this issue, and LWG should retain ownership.

AM: this is too overly broad as it also covers calling the exact same member function on a different object
STL: so you insert into a map, and copying the value triggers another insertion into a different map of the same type
GR: reentrancy seems to imply the single-threaded case, but needs to consider the multi-threaded case

Needs more wording.

Move to Open

[2015-07 Telecon Urbana]

Marshall to ping STL for updated wording.

[2016-05 email from STL]

I don't have any better suggestions than my original PR at the moment.

Previous resolution [SUPERSEDED]:

> This wording is relative to N3936.
>
> 1. Change 16.4.6.9 [reentrancy] p1 as indicated:
>
> > -1- Except where explicitly specified in this standard, it is implementation-defined which functions (including different member functions called on a single object) in the Standard C++ library may be recursively reentered.

[2021-07-29 Tim suggests new wording]

The "`this` pointer" restriction is modeled on 11.9.5 [class.cdtor] p2. It allows us to continue to specify a member function `f` as calling some other member function `g`, since any such call would use something obtained from the first member function's `this` pointer.

In all other cases, this wording disallows such "recursion on object" unless both member functions are `const` (or are treated as such for the purposes of data race avoidance). Using "access" means that we also cover direct access to the object representation, such as the following pathological example from Arthur O'Dwyer, which is now undefined:

```
std::string s = "hello world";
char *first = (char*)&s;
char *last = (char*)(&s + 1);
s.append(first, last);
```

Previous resolution [SUPERSEDED]:

> This wording is relative to N4892.
>
> 1. Add the following paragraph to 16.4.6.9 [reentrancy]:
>
> > -?- During the execution of a standard library non-static member function *F* on an object, if that object is accessed through a glvalue that is not obtained, directly or indirectly, from the `this` pointer of *F*, in a manner that can conflict (6.10.2.2 [intro.races]) with any access that *F* is permitted to perform (16.4.6.10 [res.on.data.races]), the behavior is undefined unless otherwise specified.

[2026-02-20; Tim provides new wording]

Refer to "object parameter" not "`this` pointer".

[2026-02-20; LWG telecon; Status changed: Open → Ready.]

Proposed resolution:

This wording is relative to N5032.

1. Append the following paragraph to 16.4.6.9 [reentrancy]:

> -?- During the execution of a standard library non-static member function *F* on an object, if that object is accessed through a glvalue that is not obtained, directly or indirectly, from the object parameter of *F*, in a manner that can conflict (6.10.2.2 [intro.races]) with any access that *F* is permitted to perform (16.4.6.10 [res.on.data.races]), the behavior is undefined unless otherwise specified.

2746. Inconsistency between requirements for `emplace` between `optional` and `variant`

Section: 22.5.3.4 [optional.assign], 22.6.3.5 [variant.mod], 22.7.4.4 [any.modifiers] **Status:** Ready **Submitter:** Richard Smith **Opened:** 2016-07-13 **Last modified:** 2025-12-05

Priority: 3

Discussion:

Referring to N4604:

In [optional.object.assign]: `emplace` (normal form) has a Requires that the construction works.

> *Requires:* `is_constructible_v<T, Args&&...>` is `true`.

`emplace` (`initializer_list` form) has a SFINAE condition:

> *Remarks:* […] unless `is_constructible_v<T, initializer_list<U>&, Args&&...>` is `true`.

In 22.7.4.4 [any.modifiers]: `emplace` (normal form) has a Requires that the construction works:

> *Requires:* `is_constructible_v<T, Args...>` is `true`.

`emplace` (`initializer_list` form) has a SFINAE condition:

> *Remarks:* […] unless `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

In 22.6.3.5 [variant.mod]: `emplace` (`T`, normal form) has a SFINAE condition:

> *Remarks:* […] unless `is_constructible_v<T, Args...>` is `true`, and `T` occurs exactly once in `Types...`.

`emplace` (Idx, normal form) has a *both* a Requires and a SFINAE condition:

> *Requires:* `I < sizeof...(Types)`
>
> *Remarks:* […] unless `is_constructible_v<T, Args...>` is `true`, and `T` occurs exactly once in `Types...`.

`emplace` (`T`, `initializer_list` form) has a SFINAE condition:

> *Remarks:* […] unless `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`, and `T` occurs exactly once in `Types...`.

`emplace` (Idx, `initializer_list` form) has a both a Requires and a SFINAE condition:

> *Requires:* `I < sizeof...(Types)`
>
> *Remarks:* […] unless `is_constructible_v<T, Args...>` is true, and `T` occurs exactly once in `Types...`.

Why the inconsistency? Should all the cases have a SFINAE requirement?

I see that variant has an additional requirement (`T` occurs exactly once in `Types...`), but that only agues that it must be a SFINAE condition — doesn't say that the other cases (any/variant) should not.

`map`/`multimap`/`unordered_map`/`unordered_multimap` have SFINAE'd versions of `emplace` that don't take `initializer_lists`, but they don't have any `emplace` versions that take ILs.

Suggested resolution:

Add SFINAE requirements to `optional::emplace(Args&&... args)` and `any::emplace(Args&&... args);`

[2016-08 Chicago]

During issue prioritization, people suggested that this might apply to `any` as well.

Ville notes that 2746(i), 2754(i) and 2756(i) all go together.

[2020-05-10; Daniel comments and provides wording]

The inconsistency between the two `any::emplace` overloads have been removed by resolving LWG 2754(i) to use *Constraints:* elements. The last Mandating paper (P1460R1), adopted in Prague, changed the *Requires:* elements for `variant::emplace`, `"I < sizeof...(Types)"` to *Mandates:*, but that paper was focused on fixing inappropriate preconditions, not searching for consistency here. Given that the `in_place_index_t` constructors of `variant` uses SFINAE-conditions for this form of static precondition violation, I recommend that its `emplace` functions use the same style, which would bring them also in consistency with their corresponding type-based `emplace` forms that are *Mandates:*-free but delegate to the index-based forms.

Previous resolution [SUPERSEDED]:

This wording is relative to N4861.

1. Modify 22.5.3.4 [optional.assign], as indicated:

```
template<class... Args> T& emplace(Args&&... args);
```

-29- ~~*Mandates*~~*Constraints*: `is_constructible_v<T, Args...>` is `true`.

[…]

```
template<class U, class... Args> T& emplace(initializer_list<U> il, Args&&... args);
```

-35- *Constraints:* `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

[…]

2. Modify 22.6.3.5 [variant.mod], as indicated:

```
template<class T, class... Args> T& emplace(Args&&... args);
```

-1- *Constraints:* `is_constructible_v<T, Args...>` is `true`, and `T` occurs exactly once in `Types`.

[…]

```
template<class T, class U, class... Args> T& emplace(initializer_list<U> il, Args&&... args);
```

-3- *Constraints:* `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`, and `T` occurs exactly once in `Types`.

[…]

```
template<size_t I, class... Args>
  variant_alternative_t<I, variant<Types...>>& emplace(Args&&... args);
```

~~-5- *Mandates:* `I < sizeof...(Types)`.~~

-6- *Constraints:* `is_constructible_v<`T_I`, Args...>` is `true` and `I < sizeof...(Types)` is `true`.

[…]

```
template<size_t I, class U, class... Args>
  variant_alternative_t<I, variant<Types...>>& emplace(initializer_list<U> il, Args&&... args);
```

~~-12- *Mandates:* `I < sizeof...(Types)`.~~

-13- *Constraints:* `is_constructible_v<`T_I`, initializer_list<U>&, Args...>` is `true` and `I < sizeof...(Types)` is `true`.

[…]

[2025-12-05; Jonathan provides new wording]

There's no need to use SFINAE to check if a `variant` with N alternatives allows you to emplace the N+1th alternative.

[2025-12-05; LWG telecon. Moved to Ready]

Proposed resolution:

This wording is relative to N5014.

1. Modify 22.5.3.4 [optional.assign], as indicated:

```
template<class... Args> T& emplace(Args&&... args);
```

-29- ~~*Mandates*~~*Constraints*: `is_constructible_v<T, Args...>` is `true`.

[…]

```
template<class U, class... Args> T& emplace(initializer_list<U> il, Args&&... args);
```

-35- *Constraints:* `is_constructible_v<T, initializer_list<U>&, Args...>` is `true`.

[…]

3504. `condition_variable::wait_for` is overspecified

Section: 32.7.4 [thread.condition.condvar] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2020-11-18 **Last modified:** 2026-02-20

Priority: 3

View all other issues in [thread.condition.condvar].

Discussion:

32.7.4 [thread.condition.condvar] p24 says:

> *Effects:* Equivalent to: `return wait_until(lock, chrono::steady_clock::now() + rel_time);`

This is overspecification, removing implementer freedom to make `cv.wait_for(duration<float>(1))` work accurately.

The type of `steady_clock::now() + duration<float>(n)` is `time_point<steady_clock, duration<float, steady_clock::period>>`. If the steady clock's period is `std::nano` and its epoch is the time the system booted, then in under a second a 32-bit `float` becomes unable to exactly represent those `time_points`! Every second after boot makes `duration<float, nano>` less precise.

This means that adding a `duration<float>` to a `time_point` (or `duration`) measured in nanoseconds is unlikely to produce an accurate value. Either it will round down to a value less than `now()`, or round up to one greater than `now() + 1s`. Either way, the `wait_for(rel_time)` doesn't wait for the specified time, and users think the implementation is faulty.

A possible solution is to use `steady_clock::now() + ceil<steady_clock::duration>(rel_time)` instead. This converts the relative time to a suitably large integer, and then the addition is not affected by floating-point rounding errors due to the limited precision of 32-bit `float`. Libstdc++ has been doing this for nearly three years. Alternatively, the standard could just say that the relative timeout is converted to an absolute timeout measured against the steady clock, and leave the details to the implementation. Some implementations might not be affected by the problem (e.g. if the steady clock measures milliseconds, or processes only run for a few seconds and use the process start as the steady clock's epoch).

This also affects the other overload of `condition_variable::wait_for`, and both overloads of `condition_variable_any::wait_for`.

[2020-11-29; Reflector prioritization]

Set priority to 3 during reflector discussions.

[2024-06-18; Jonathan adds wording]

Previous resolution [SUPERSEDED]:

This wording is relative to N4981.

1. Modify 32.7.1 [thread.condition.general] as indicated, adding a new paragraph to the end:

> -6- The definitions in 32.7 [thread.condition] make use of the following exposition-only function:
>
> ```
> template<class Dur>
>   chrono::steady_clock::time_point rel-to-abs(const Dur& rel_time)
>   { return chrono::steady_clock::now() + chrono::ceil<chrono::steady_clock::duration>(rel_time); }
> ```

2. Modify 32.7.4 [thread.condition.condvar] as indicated:

> ```
> template<class Rep, class Period>
>   cv_status wait_for(unique_lock<mutex>& lock,
>                      const chrono::duration<Rep, Period>& rel_time);
> ```
>
> -23- *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either [...]
>
> -24- *Effects*: Equivalent to:
>
> `return wait_until(lock, ~~chrono::steady_clock::now() + rel_time~~ rel-to-abs(rel_time));`
>
> [...]
>
> ```
> template<class Rep, class Period, class Predicate>
>   cv_status wait_for(unique_lock<mutex>& lock,
>                      const chrono::duration<Rep, Period>& rel_time,
>                      Predicate pred);
> ```
>
> -35- *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either [...]
>
> -36- *Effects*: Equivalent to:
>
> `return wait_until(lock, ~~chrono::steady_clock::now() + rel_time~~ rel-to-abs(rel_time), std::move(pred));`
>
> [*Note 8*: There is no blocking if `pred()` is initially `true`, even if the timeout has already expired. — *end note*]

3. Modify 32.7.5.2 [thread.condvarany.wait] as indicated:

> ```
> template<class Lock, class Rep, class Period>
>   cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time);
> ```
>
> -11- *Effects*: Equivalent to:
>
> `return wait_until(lock, ~~chrono::steady_clock::now() + rel_time~~ rel-to-abs(rel_time));`

[...]

```
template<class Lock, class Rep, class Period, class Predicate>
  cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time, Predicate pred);
```

-19- *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time rel-to-abs(rel_time),
std::move(pred));
```

[2026-02-20; Jonathan provides new wording]

Need to add another change for the interruptible `wait_for`.

[2026-02-20; LWG telecon; Status changed: New → Ready.]

Proposed resolution:

This wording is relative to N5032.

1. Modify 32.7.1 [thread.condition.general] as indicated, adding a new paragraph to the end:

-6- The definitions in 32.7 [thread.condition] make use of the following exposition-only function:

```
template<class Dur>
  chrono::steady_clock::time_point rel-to-abs(const Dur& rel_time)
  { return chrono::steady_clock::now() + chrono::ceil<chrono::steady_clock::duration>(rel_time); }
```

2. Modify 32.7.4 [thread.condition.condvar] as indicated:

```
template<class Rep, class Period>
  cv_status wait_for(unique_lock<mutex>& lock,
                     const chrono::duration<Rep, Period>& rel_time);
```

-23- *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either [...]

-24- *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time rel-to-abs(rel_time));
```

[...]

```
template<class Rep, class Period, class Predicate>
  cv_status wait_for(unique_lock<mutex>& lock,
                     const chrono::duration<Rep, Period>& rel_time,
                     Predicate pred);
```

-35- *Preconditions*: `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either [...]

-36- *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time rel-to-abs(rel_time), std::move(pred));
```

[*Note 8*: There is no blocking if `pred()` is initially `true`, even if the timeout has already expired. — *end note*]

3. Modify 32.7.5.2 [thread.condvarany.wait] as indicated:

```
template<class Lock, class Rep, class Period>
  cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time);
```

-11- *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time rel-to-abs(rel_time));
```

[...]

```
template<class Lock, class Rep, class Period, class Predicate>
  cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time, Predicate pred);
```

-19- *Effects*: Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time rel-to-abs(rel_time), std::move(pred));
```

4. Modify 32.7.5.3 [thread.condvarany.intwait] as indicated:

```
template<class Lock, class Rep, class Period, class Predicate>
```

```
bool wait_for(Lock& lock, stop_token stoken,
              const chrono::duration<Rep, Period>& rel_time, Predicate pred);
```

-13- *Effects*: Equivalent to:

```
return wait_until(lock, std::move(stoken), chrono::steady_clock::now() + rel_time  rel-to-abs(rel_time),
std::move(pred));
```

3599. The `const` overload of `lazy_split_view::begin` should be constrained by `const Pattern`

Section: 25.7.16.2 [range.lazy.split.view] **Status:** Ready **Submitter:** Hewill Kang **Opened:** 2021-09-23 **Last modified:** 2026-01-16

Priority: 3

View other active issues in [range.lazy.split.view].

View all other issues in [range.lazy.split.view].

Discussion:

Consider the following code snippet:

```
#include <ranges>

int main() {
  auto p = std::views::iota(0)
         | std::views::take(1)
         | std::views::reverse;
  auto r = std::views::single(42)
         | std::views::lazy_split(p);
  auto f = r.front();
}
```

`r.front()` is ill-formed even if `r` is a `forward_range`.

This is because the `const` overload of `lazy_split_view::begin` is not constrained by the `const Pattern`, which makes it still well-formed in such cases. When the `const` overload of `view_interface<lazy_split_view<V, Pattern>>::front` is instantiated, the `subrange{parent_->pattern_}` inside `lazy_split_view::outer-iterator<true>::operator++()` will cause a hard error since `const Pattern` is not a `range`.

[2021-09-24; Daniel comments]

This issue is related to LWG 3592[(i)].

[2021-10-14; Reflector poll]

Set priority to 3 after reflector poll.

[2026-01-16; LWG telecon. Move to Ready]

Proposed resolution:

This wording is relative to N4892.

1. Modify 25.7.16.2 [range.lazy.split.view], class template `lazy_split_view` synopsis, as indicated:

```
namespace std::ranges {

  […]

  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
            (forward_range<V> || tiny-range<Pattern>)
  class lazy_split_view : public view_interface<lazy_split_view<V, Pattern>> {
  private:
    […]
  public:
    […]

    constexpr auto begin() {
      […]
    }

    constexpr auto begin() const requires forward_range<V> && forward_range<const V> &&
                                          forward_range<const Pattern>{
      return outer-iterator<true>{*this, ranges::begin(base_)};
    }

    constexpr auto end() requires forward_range<V> && common_range<V> {
      […]
    }

    constexpr auto end() const {
      if constexpr (forward_range<V> && forward_range<const V> && common_range<const V> &&
```

```
                    forward_range<const Pattern>)
        return outer-iterator<true>{*this, ranges::end(base_)};
      else
        return default_sentinel;
    }
  };

  […]

}
```

4290. Missing *Mandates* clauses on `is_sufficiently_aligned`

Section: 20.2.5 [ptr.align] **Status:** Ready **Submitter:** Damien Lebrun-Grandie **Opened:** 2025-07-03 **Last modified:** 2026-01-30

Priority: 2

View other active issues in [ptr.align].

View all other issues in [ptr.align].

Discussion:

`is_sufficiently_aligned` should mandate that the alignment template argument is a power of two and that it is greater equal to the byte alignment of its type template argument.

In 20.2.5 [ptr.align] `is_sufficiently_aligned` has no *Mandates* element. It is an oversight that we realized when implementing P2897R7 into libc++ (in https://github.com/llvm/llvm-project/pull/122603). The function template was originally proposed as a static member function of the `aligned_accessor` class template which has these two *Mandates* clauses and therefore applied (see 23.7.3.5.4.1 [mdspan.accessor.aligned.overview] p1). It revision P2897R4, `is_sufficiently_aligned` was moved out the class template definition to become the free function memory helper that was voted into C++26 but the *Mandates* were lost in the process.

We propose to correct that oversight and reintroduce the following *Mandates* clauses right above 20.2.5 [ptr.align] p10.

[2025-10-23; Reflector poll.]

Set priority to 2 after reflector poll.

The *Mandates* of `Alignment >= alignof(T)` was put into question, as being too restrictive for general purpose free function.

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

1. Modify 20.2.5 [ptr.align] as indicated:

```
template<size_t Alignment, class T>
  bool is_sufficiently_aligned(T* ptr);
```

-?- *Mandates*:

(?.1) — `Alignment` is a power of two, and

(?.2) — `Alignment >= alignof(T)` is `true`.

-10- *Preconditions*: `p` points to an object `X` of a type similar (7.3.6 [conv.qual]) to `T`.

-11- *Returns*: `true` if `X` has alignment at least `Alignment`, otherwise `false`.

-12- *Throws*: Nothing.

[2026-01-30; Jonathan provides new wording]

[2026-01-30; LWG telecon. Status → Ready.]

Proposed resolution:

This wording is relative to N5032.

1. Modify 20.2.5 [ptr.align] as indicated:

```
template<size_t Alignment, class T>
  bool is_sufficiently_aligned(T* ptr);
```

-?- *Mandates*: `Alignment` is a power of two.

-10- *Preconditions*: `p` points to an object `X` of a type similar (7.3.6 [conv.qual]) to `T`.

-11- *Returns*: `true` if `X` has alignment at least `Alignment`, otherwise `false`.

-12- *Throws*: Nothing.

4453. `atomic_ref<cv T>::required_alignment` should be the same as for `T`

Section: 32.5.7.2 [atomics.ref.ops] **Status:** New **Submitter:** Jonathan Wakely **Opened:** 2025-11-06 **Last modified:** 2026-02-27

Priority: 2

View other active issues in [atomics.ref.ops].

View all other issues in [atomics.ref.ops].

Discussion:

When P3323R1 fixed support for `atomic_ref<cv T>` we didn't consider that an implementation could define `required_alignment` to be different from `atomic_ref<T>::required_alignment`. For example, a processor could support atomic loads on any `int` object, but could require a greater alignment for atomic stores (and read-modify-write and modify-write operations). So the `required_alignment` could be `alignof(int)` for `const int` but `2 * alignof(int)` for `int`, because the latter is needed for stores.

A malicious implementation could even define `required_alignment` to be greater for the `const T` specialization, which would make it undefined to use the new converting constructor added by P3860R1.

We should constrain implementations to use the same `required_alignment` for cv `T`, so that users can use `atomic_ref<const T>` on any object that is referenced by a `atomic_ref<T>`.

[2026-01-16; Reflector poll.]

Set priority to 2 after reflector poll.

"Should be broaded to include all similar types, not just top-level *cv*."

"Should we have the same guarantee for `is_always_lock_free`? Would allow checking if `atomic<volatile T>` is usable by checking `atomic<T>`."

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify [atomics.refs.ops], as indicated:

```
static constexpr size_t required_alignment;
```

-1- The alignment required for an object to be referenced by an atomic reference, which is at least `alignof(T)`. The value of `required_alignment` is the same as `atomic_ref<remove_cv_t<T>>::required_alignment`

-2- [*Note 1*: Hardware could require an object referenced by an `atomic_ref` to have stricter alignment (6.8.3 [basic.align]) than other objects of type `T`. Further, whether operations on an `atomic_ref` are lock-free could depend on the alignment of the referenced object. For example, lock-free operations on `std::complex<double>` could be supported only if aligned to `2*alignof(double)`. — *end note*]

[2026-02-16; Tomasz provides updated resolution.]

Guarantee that `required_alignment` and `is_always_lock_free` values are the same for any similar types.

We define equality to `atomic_ref<remove_cv_t<U>`, as `volatile` atomics may be ill-formed.

Proposed resolution:

This wording is relative to N5032.

1. Modify [atomics.refs.ops], as indicated:

```
static constexpr size_t required_alignment;
```

-1- The alignment required for an object to be referenced by an atomic reference, which is at least `alignof(T)`. For any type `U` similar to `T`, the value of `required_alignment` is the same as `atomic_ref<remove_cv_t<U>>::required_alignment`.

-2- [*Note 1*: Hardware could require an object referenced by an `atomic_ref` to have stricter alignment (6.8.3 [basic.align]) than other objects of type `T`. Further, whether operations on an `atomic_ref` are lock-free could depend on the alignment of the referenced object. For example, lock-free operations on `std::complex<double>` could be supported only if aligned to `2*alignof(double)`. — *end note*]

```
static constexpr bool is_always_lock_free;
```

-3- The static data member `is_always_lock_free` is `true` if the `atomic_ref` type's operations are always lock-free, and false otherwise. For any type `U` similar to `T` the value of `is_always_lock_free` is the same as `atomic_ref<remove_cv_t<U>>::is_always_lock_free`.

4454. `assert` should forbid `co_await` and `co_yield`

Section: 19.3.3 [assertions.assert] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2025-11-06 **Last modified:** 2026-03-06

Priority: 1

Discussion:

Addresses GB 05-129

The new implementation of `assert` introduced by P2264R7 might require something like `sizeof(bool(__VA_ARGS__))`. Using something like `assert((co_yield 1, true))` would be ill-formed if the implementation of `assert` uses `sizeof`, because `co_yield` cannot appear in an unevaluated context.

The specification for `assert` should forbid using any constructs which are not valid in unevaluated contexts, unless we are certain that the new `assert` requirements can be implemented without such tricks using unevaluated contexts. "Ill-formed; no diagnostic required" seems appropriate.

[2026-03-06 Tim provides wording]

[2026-03-06 LWG telecon; move to Ready]

Proposed resolution:

This wording is relative to N5032.

Modify 19.3.3 [assertions.assert] as indicated:

> -3- If `__VA_ARGS__` does not expand to ~~an~~a well-formed *assignment-expression*, the program is ill-formed. If such an *assignment-expression* is ill-formed when treated as an unevaluated operand (7.6.2.4 [expr.await], 7.6.17 [expr.yield]), the program is ill-formed, no diagnostic required.

4469. Names of parameters of addressable function shall remain unspecified

Section: 16.4.5.2.1 [namespace.std] **Status:** Ready **Submitter:** Tomasz Kamiński **Opened:** 2025-11-14 **Last modified:** 2026-01-16

Priority: 2

View other active issues in [namespace.std].

View all other issues in [namespace.std].

Discussion:

The wording in 16.4.5.2.1 [namespace.std] p7 guarantees that reflection of the addressable function can be reliably produced. With the addition of the function parameter name, it is possible to access the parameter name of such functions:

```
constexpr auto p = parameters_of(^^std::endl)[0];
static_assert( identifier_of(p) == "os" ); // guaranteed?
```

We should clarify that parameter names used by standard library implementation remain unspecified, by making result of `has_identifier` and `identifier_of` unspecified on reflection of such parameter.

[2026-01-16; Reflector poll.]

Set priority to 2 after reflector poll.

[2026-01-16; LWG telecon. Move to Ready]

Proposed resolution:

This wording is relative to N5014.

1. Modify 16.4.5.2.1 [namespace.std], as indicated:

> Let *F* denote a standard library function or function template. Unless *F* is designated an addressable function, it is unspecified if or how a reflection value designating the associated entity can be formed. For any value *p* of type `meta::info` that represents a reflection of a parameter of *F*, it is unspecified if `has_identifier(`*p*`)` returns `true` or `false`, and if `meta::has_identifier(`*p*`)` is `true`, then the NTMBS produced by `meta::identifier_of(`*p*`)` and `meta::u8identifier_of(`*p*`)` is unspecified.

4472. `std::atomic_ref<const T>` can be constructed from temporaries

Section: 32.5.7 [atomics.ref.generic] **Status:** Ready **Submitter:** Jiang An **Opened:** 2025-11-11 **Last modified:** 2025-12-05

Priority: Not Prioritized

View all other issues in [atomics.ref.generic].

Discussion:

`std::atomic_ref<T>` has a constructor taking `T&`, so when `T` is a `const` but not `volatile` object type, the constructor parameter can be bound to a temporary expression, which doesn't seem to make sense. Even after P3860R1, explicitly constructing `std::atomic_ref<const T>` from `std::atomic_ref<volatile T>` can be well-formed with the undesired semantics when it is well-formed to instantiate `std::atomic_ref<volatile T>` and its operator `T` conversion function, because the construction calls the conversion function and creates a temporary object. Probably it's better to disallow such reference binding.

[2025-12-05; LWG telecon. Moved to Ready]

Proposed resolution:

This wording is relative to N5014 after application of P3860R1.

> [*Drafting note*: The deleted overloads were mirrored from the design of `reference_wrapper` before LWG 2993[(i)]. As these overloads don't participate in implicit conversion, I don't think there will be any similar issue introduced.]

1. Modify 32.5.7.1 [atomics.ref.generic.general], primary class template `atomic_ref` synopsis, as indicated:

```
namespace std {
  template<class T> struct atomic_ref {
  private:
    T* ptr;              // exposition only
  public:
    […]
    constexpr explicit atomic_ref(T&);
    explicit atomic_ref(T&&) = delete;
    constexpr atomic_ref(const atomic_ref&) noexcept;
    template<class U>
      constexpr atomic_ref(const atomic_ref<U>&) noexcept;
    […]
  };
}
```

2. Modify 32.5.7.3 [atomics.ref.int], class template `atomic_ref` *integral-type* specialization synopsis, as indicated:

```
namespace std {
  template<> struct atomic_ref<integral-type> {
  private:
    integral-type* ptr;              // exposition only
  public:
    […]
    constexpr explicit atomic_ref(integral-type&);
    explicit atomic_ref(integral-type&&) = delete;
    constexpr atomic_ref(const atomic_ref&) noexcept;
    template<class U>
      constexpr atomic_ref(const atomic_ref<U>&) noexcept;
    […]
  };
}
```

3. Modify 32.5.7.4 [atomics.ref.float], class template `atomic_ref` *floating-point-type* specialization synopsis, as indicated:

```
namespace std {
  template<> struct atomic_ref<floating-point-type> {
  private:
    floating-point-type* ptr;              // exposition only
  public:
    […]
    constexpr explicit atomic_ref(floating-point-type&);
    explicit atomic_ref(floating-point-type&&) = delete;
    constexpr atomic_ref(const atomic_ref&) noexcept;
    template<class U>
      constexpr atomic_ref(const atomic_ref<U>&) noexcept;
    […]
  };
}
```

4. Modify 32.5.7.5 [atomics.ref.pointer], class template `atomic_ref` *pointer-type* specialization synopsis, as indicated:

```
namespace std {
  template<> struct atomic_ref<pointer-type> {
  private:
    pointer-type* ptr;              // exposition only
  public:
    […]
    constexpr explicit atomic_ref(pointer-type&);
    explicit atomic_ref(pointer-type&&) = delete;
    constexpr atomic_ref(const atomic_ref&) noexcept;
    template<class U>
      constexpr atomic_ref(const atomic_ref<U>&) noexcept;
    […]
  };
}
```

4486. *integral-constant-like* and *constexpr-wrapper-like* exposition-only concept duplication

Section: 23.7.2.1 [span.syn], 29.10.2 [simd.expos] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2025-12-12 **Last modified:** 2025-12-12

Priority: Not Prioritized

View all other issues in [span.syn].

Discussion:

Addresses US 151-242

NB comment:

> We have two exposition-only concepts for similar things (*integral-constant-like* and *constant-wrapper-like*). The former can be expressed in terms of the latter.
> Proposed change: Move *constant-wrapper-like* to the library introduction and update *integral-constant-like* to use it.

[2025-12-12; LWG telecon; Status changed: New → Ready.]

Proposed resolution:

This wording is relative to N5014.

1. Modify 16.3.3.2 [expos.only.entity] by moving the concept (unchanged) from 29.10.2 [simd.expos]:

```
  template<class T, class U = T>
  using synth-three-way-result =                        // exposition only
    decltype(synth-three-way(declval<T&>(), declval<U&>()));

  template<class T>
    concept constexpr-wrapper-like =                    // exposition only
      convertible_to<T, decltype(T::value)> &&
      equality_comparable_with<T, decltype(T::value)> &&
      bool_constant<T() == T::value>::value &&
      bool_constant<static_cast<decltype(T::value)>(T()) == T::value>::value;
}
```

2. Modify 23.7.2.1 [span.syn] as indicated:

```
template<class T>
  concept integral-constant-like =                    // exposition only
    is_integral_v<remove_cvref_t<decltype(T::value)>> &&
    !is_same_v<bool, remove_const_t<decltype(T::value)>> &&
    constexpr-wrapper-like<T>;
    convertible_to<T, decltype(T::value)> &&
    equality_comparable_with<T, decltype(T::value)> &&
    bool_constant<T() == T::value>::value &&
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value>::value;
```

3. Modify 29.10.2 [simd.expos] as indicated:

```
template<class T>
  concept constexpr-wrapper-like =                    // exposition only
    convertible_to<T, decltype(T::value)> &&
    equality_comparable_with<T, decltype(T::value)> &&
    bool_constant<T() == T::value>::value &&
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value>::value;
```

4492. `std::generate` and `std::ranges::generate` wording is unclear for parallel algorithms

Section: 26.7.7 [alg.generate] **Status:** Ready **Submitter:** Ruslan Arutyunyan **Opened:** 2025-12-12 **Last modified:** 2026-01-30

Priority: 2

Discussion:

`std::generate` and `std::ranges::generate` are confusing for parallel algorithms. For both of those *Effects* says "Assigns the result of successive evaluations of `gen()` through each iterator in the range `[first, first + N)`." The word "successive" is confusing when we talk about parallelism. This wording was the preexisting one; P3179 "Parallel Range Algorithms" didn't modify that, so the problem existed even before.

Another problem is that there is nothing about what is allowed to do with the `Generator` template parameter type for C++17 parallel `generate`. Intel, NVIDIA, and GNU libstdc++ implementations do multiple copies of `Generator` object to maximize parallelism, while it's not technically allowed if I am reading the standard correctly. But without `Generator` being copyable we have point of contention and extra synchronization (or we put synchronization part as users' responsibility, which is also not obvious).

There is no clear solution right away. We could try to fix the wording for both ranges and C++17 parallel generate but it seems like it requires extra effort because we need to at least verify the new behavior with LLVM libc++ implementation if we want to make `Generator` copyable; libc++ currently does not create per-thread copy of `gen` object. Perhaps, the best strategy for now is to remove `ranges::generate` and `ranges::generate_n` and return them back in C++29 time frame when we figure out the proper fix.

[2026-01-16; Reflector poll.]

Set priority to 2 after reflector poll.

[2026-01-30; LWG telecon. Status → Ready.]

Proposed resolution:

This wording is relative to N5032.

1. Modify 26.4 [algorithm.syn], header `<algorithm>` synopsis, as indicated:

```
[…]
// 26.7.7 [alg.generate], generate
template<class ForwardIterator, class Generator>
  constexpr void generate(ForwardIterator first, ForwardIterator last,
                          Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Generator>
  void generate(ExecutionPolicy&& exec,   // freestanding-deleted, see 26.3.5 [algorithms.parallel.overloads]
                ForwardIterator first, ForwardIterator last,
                Generator gen);
template<class OutputIterator, class Size, class Generator>
  constexpr OutputIterator generate_n(OutputIterator first, Size n, Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Size, class Generator>
  ForwardIterator generate_n(ExecutionPolicy&& exec,   // freestanding-deleted, see 26.3.5 [algorithms.parallel.overloads]
                             ForwardIterator first, Size n, Generator gen);

namespace ranges {
  template<input_or_output_iterator O, sentinel_for<O> S, copy_constructible F>
    requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
    constexpr O generate(O first, S last, F gen);
  template<class R, copy_constructible F>
    requires invocable<F&> & output_range<R, invoke_result_t<F&>>
    constexpr borrowed_iterator_t<R> generate(R&& r, F gen);
  template<input_or_output_iterator O, copy_constructible F>
    requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
    constexpr O generate_n(O first, iter_difference_t<O> n, F gen);
  template<execution-policy Ep, random_access_iterator O, sized_sentinel_for<O> S,
           copy_constructible F>
    requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
    O generate(Ep&& exec, O first, S last, F gen); // freestanding-deleted
  template<execution_policy Ep, sized_random_access_range R, copy_constructible F>
    requires invocable<F&> & indirectly_writable<iterator_t<R>, invoke_result_t<F&>>
    borrowed_iterator_t<R> generate(Ep&& exec, R&& r, F gen); // freestanding-deleted
  template<execution_policy Ep, random_access_iterator O, copy_constructible F>
    requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
    O generate_n(Ep&& exec, O first, iter_difference_t<O> n, F gen); // freestanding-deleted
}
[…]
```

2. Modify 26.7.7 [alg.generate] as indicated:

```
template<class ForwardIterator, class Generator>
  constexpr void generate(ForwardIterator first, ForwardIterator last,
                          Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Generator>
  void generate(ExecutionPolicy&& exec,
                ForwardIterator first, ForwardIterator last,
                Generator gen);

template<class OutputIterator, class Size, class Generator>
  constexpr OutputIterator generate_n(OutputIterator first, Size n, Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Size, class Generator>
  ForwardIterator generate_n(ExecutionPolicy&& exec,
                             ForwardIterator first, Size n, Generator gen);

template<input_or_output_iterator O, sentinel_for<O> S, copy_constructible F>
  requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
  constexpr O ranges::generate(O first, S last, F gen);
template<class R, copy_constructible F>
  requires invocable<F&> & output_range<R, invoke_result_t<F&>>
  constexpr borrowed_iterator_t<R> ranges::generate(R&& r, F gen);
template<input_or_output_iterator O, copy_constructible F>
  requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
  constexpr O ranges::generate_n(O first, iter_difference_t<O> n, F gen);
template<execution-policy Ep, random_access_iterator O, sized_sentinel_for<O> S,
         copy_constructible F>
  requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
  O ranges::generate(Ep&& exec, O first, S last, F gen);
template<execution_policy Ep, sized_random_access_range R, copy_constructible F>
  requires invocable<F&> & indirectly_writable<iterator_t<R>, invoke_result_t<F&>>
  borrowed_iterator_t<R> ranges::generate(Ep&& exec, R&& r, F gen);
template<execution_policy Ep, random_access_iterator O, copy_constructible F>
  requires invocable<F&> & indirectly_writable<O, invoke_result_t<F&>>
  O ranges::generate_n(Ep&& exec, O first, iter_difference_t<O> n, F gen);
```

-1- Let *N* be `max(0, n)` for the `generate_n` algorithms, and `last - first` for the `generate` algorithms.

[…]

4496. Precedes vs Reachable in [meta.reflection]

Section: 21.4 [meta.reflection] **Status:** Ready **Submitter:** Daniel Katz **Opened:** 2025-12-11 **Last modified:** 2026-02-20

Priority: 2

Discussion:

Discussion on the Core mailing list surfaced a handful of places in 21.4 [meta.reflection] that use the "precedes" relation (defined in 6.5 [basic.lookup] and primarily used for name lookup) when the "reachable" relation (defined in 10.7 [module.reach]) is really what we want.

[2026-02-18; Reflector poll.]

Set priority to 2 after reflector poll.

[2026-02-20; LWG telecon; Status changed: New → Ready.]

Proposed resolution:

This wording is relative to N5032.

1. Modify 21.4.6 [meta.reflection.names] as indicated:

```
consteval bool has_identifier(info r);
```

-1- *Returns*:

(1.1) — […]

[…]

(1.8) — Otherwise, if `r` represents the parameter *P* of a function *F*, then let *S* be the set of declarations, ignoring any explicit instantiations, that ~~precede some~~are reachable from a point in the evaluation context and that declare either *F* or a templated function of which *F* is a specialization; […]

[…]

```
consteval string_view identifier_of(info r);
consteval u8string_view u8identifier_of(info r);
```

-2- Let *E* be UTF-8 for `u8identifier_of`, and otherwise the ordinary literal encoding.

-3- *Returns*: An NTMBS, encoded with *E*, determined as follows:

(3.1) — […]

(3.2) — […]

(3.3) — Otherwise, if `r` represents the parameter *P* of a function *F*, then let *S* be the set of declarations, ignoring any explicit instantiations, that ~~precede some~~are reachable from a point in the evaluation context and that declare either *F* or a templated function of which *F* is a specialization; the name that was introduced by a declaration in *S* for the parameter corresponding to *P*.

[…]

2. Modify 21.4.7 [meta.reflection.queries] as indicated:

```
consteval info type_of(info r);
```

-2- *Returns*:

(2.1) — […]

[…]

(2.4) — Otherwise, if `r` represents an enumerator *N* of an enumeration *E*, then:

(2.4.1) — If *E* is defined by a declaration *D* that ~~precedes~~is reachable from a point *P* in the evaluation context and *P* does not occur within an *enum-specifier* of *D*, then a reflection of *E*.

(2.4.2) — Otherwise, a reflection of the type of *N* prior to the closing brace of the *enum-specifier* as specified in 9.8.1 [dcl.enum].

[…]

[…]

```
consteval bool has_default_argument(info r);
```

-41- *Returns*: If `r` represents a parameter *P* of a function *F*, then:

(41.1) — If *F* is a specialization of a templated function *T*, then `true` if there exists a declaration *D* of *T* that ~~precedes some~~is reachable from a point in the evaluation context and *D* specifies a default argument for the parameter of *T* corresponding to *P*. Otherwise, `false`.

(41.2) — Otherwise, if there exists a declaration *D* of *F* that ~~precedes some~~is reachable from a point in the evaluation context and *D* specifies a default argument for *P*, then `true`.

3. Modify 21.4.18 [meta.reflection.annotation] as indicated:

```
consteval vector<info> annotations_of(info item);
```

-1- Let *E* be […]

-2- *Returns*: A `vector` containing all of the reflections *R* representing each annotation applying to each declaration of *E* that ~~precedes~~is reachable from either ~~some~~a point in the evaluation context (7.7 [expr.const]) or a point immediately following the *class-specifier* of the outermost class for which such a point is in a complete-class context. […]

4499. `flat_set::insert_range` specification may be problematic

Section: 23.6.11.5 [flat.set.modifiers], 23.6.12.5 [flat.multiset.modifiers] **Status:** Ready **Submitter:** Hewill Kang **Opened:** 2025-12-22 **Last modified:** 2026-02-20

Priority: 2

Discussion:

The function adds elements via:

```
ranges::for_each(rg, [&](auto&& e) {
  c.insert(c.end(), std::forward<decltype(e)>(e));
});
```

Here, `e` is an element of the input range.

However, this can lead to ambiguity when `e` can also be converted to `initializer_list`, as `vector::insert` has an overload of `insert(const_iterator, initializer_list<T>)`.

[2026-02-18; Reflector poll.]

Set priority to 2 after reflector poll.

Pessimizing a very common case (matching type), for uncommon one.

[2026-02-20; LWG telecon; Status changed: New → Ready.]

Implementations can optimize away the extra move for the case where the range's value type is already the same as the flat set's value type.

Proposed resolution:

This wording is relative to N5032.

1. Modify 23.6.11.5 [flat.set.modifiers] as indicated:

```
template<container-compatible-range<value_type> R>
  constexpr void insert_range(R&& rg);
```

-10- *Effects*: Adds elements to *c* as if by:

```
ranges::for_each(rg, [&](value_type~~auto&&~~ e) {
  c.insert(c.end(), std::move~~std::forward<decltype(e)>~~(e));
});
```

2. Modify 23.6.12.5 [flat.multiset.modifiers] as indicated:

```
template<container-compatible-range<value_type> R>
  void insert_range(R&& rg);
```

-9- *Effects*: Adds elements to *c* as if by:

```
ranges::for_each(rg, [&](value_type~~auto&&~~ e) {
  c.insert(c.end(), std::move~~std::forward<decltype(e)>~~(e));
});
```

4510. Ambiguity of `std::ranges::advance` and `std::ranges::next` when the difference type is also a sentinel type

Section: 24.4.4.2 [range.iter.op.advance], 24.4.4.4 [range.iter.op.next] **Status:** Ready **Submitter:** Jiang An **Opened:** 2026-01-09 **Last modified:** 2026-02-27

Priority: 3

View all other issues in [range.iter.op.advance].

Discussion:

Currently, `ranges::advance` and `ranges::next` have `operator()` overloads that accept `(iterator, sentinel)` and `(iterator, difference)`. However, when the difference type of the iterator is also a sentinel type, there may be ambiguity in both overloads.

E.g. the following example is rejected when compiling with libc++ or libstdc++ (demo):

```
#include <cstddef>
#include <iterator>
#include <type_traits>

template<class T>
struct FwdIter { // triggers ADL for T
  FwdIter();

  using value_type      = std::remove_cv_t<T>;
  using difference_type = int;

  T& operator*() const;

  FwdIter& operator++();
  FwdIter operator++(int);

  friend bool operator==(const FwdIter&, const FwdIter&);
};

static_assert(std::forward_iterator<FwdIter<int>>);

struct OmniConv {
  OmniConv(const auto&);
  friend bool operator==(OmniConv, OmniConv); // found by ADL via things related to OmniConv
};

int main() {
  FwdIter<OmniConv> it{};
  std::ranges::advance(it, 0); // ambiguous
  std::ranges::next(it, 0); // ambiguous
}
```

Perhaps it would be better to ensure that calling `ranges::advance` or `ranges::next` with an iterator value and a value of the difference type is unambiguous. Note that wrapping iterators that trigger ADL for the value type like the `FwdIter` in this example are common in standard library implementations, so ambiguity can be easily raised from containers with such `OmniConv` being their element type.

[2026-02-20; Reflector poll.]

Set priority to 3 after reflector poll.

The type in the example is basically a `std::any` that additionally type-erases equality. That causes a problem for templated iterators that have their value type's namespace as an associated namespace for ADL.

Instead of allowing integers to be sentinels in general but just restricting them from being used with these overloads, we could just change the concept so that integers do not model `sentinel_for`. That's similar to what we did for `span`'s constructor which has a similar problem (though that one's more aggressive due to compatibility constraints).

Previous resolution [SUPERSEDED]:

This wording is relative to N5032.

1. Modify 24.2 [iterator.synopsis], header `<iterator>` synopsis, as indicated:

```
[…]
namespace std {
  template<class T> using with-reference = T&; // exposition only
  template<class T> concept can-reference      // exposition only
    = requires { typename with-reference<T>; };
  template<class T> concept dereferenceable    // exposition only
    = requires(T& t) {
      { *t } -> can-reference; // not required to be equality-preserving
    };

  template<class T>
    constexpr bool is-integer-like = see below;           // exposition only

  […]
  // 24.4.4 [range.iter.ops], range iterator operations
  namespace ranges {
    // 24.4.4.2 [range.iter.op.advance], ranges::advance
    template<input_or_output_iterator I>
      constexpr void advance(I& i, iter_difference_t<I> n);               // freestanding
    template<input_or_output_iterator I, sentinel_for<I>class S>
      requires (!is-integer-like<S>) && sentinel_for<S, I>
      constexpr void advance(I& i, S bound);                              // freestanding
    template<input_or_output_iterator I, sentinel_for<I> S>
      constexpr iter_difference_t<I> advance(I& i, iter_difference_t<I> n, // freestanding
```

```
                                                S bound);
    […]
    // 24.4.4.4 [range.iter.op.next], ranges::next
    template<input_or_output_iterator I>
      constexpr I next(I x);                                    // freestanding
    template<input_or_output_iterator I>
      constexpr I next(I x, iter_difference_t<I> n);            // freestanding
    template<input_or_output_iterator I, sentinel_for<I> S>
      requires (!is-integer-like<S>) && sentinel_for<S, I>
      constexpr I next(I x, S bound);                           // freestanding
    template<input_or_output_iterator I, sentinel_for<I> S>
      constexpr I next(I x, iter_difference_t<I> n, S bound); // freestanding
  }
  […]
}
```

2. Modify 24.4.4.2 [range.iter.op.advance] as indicated:

```
template<input_or_output_iterator I, sentinel_for<I> S>
  requires (!is-integer-like<S>) && sentinel_for<S, I>
  constexpr void advance(I& i, S bound);
```

-3- *Preconditions*: […]

3. Modify 24.4.4.4 [range.iter.op.next] as indicated:

```
template<input_or_output_iterator I, sentinel_for<I> S>
  requires (!is-integer-like<S>) && sentinel_for<S, I>
  constexpr I next(I x, S bound);
```

-3- *Effects*: […]

[2026-02-20; Jonathan provides new wording]

[2026-02-27; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to N5032.

1. Modify 24.3.4.7 [iterator.concept.sentinel] as indicated:

```
template<class S, class I>
  concept sentinel_for =
    semiregular<S> &&
    !is-integer-like<S> &&
    input_or_output_iterator<I> &&
    weakly-equality-comparable-with<S, I>; // see 18.5.4 [concept.equalitycomparable]
```

4511. Inconsistency between the deduction guide of `std::mdspan` taking `(data_handle_type, mapping_type, accessor_type)` and the corresponding constructor

Section: 23.7.3.6.1 [mdspan.mdspan.overview] **Status:** Ready **Submitter:** Jiang An **Opened:** 2026-01-09 **Last modified:** 2026-02-20

Priority: Not Prioritized

View other active issues in [mdspan.mdspan.overview].

View all other issues in [mdspan.mdspan.overview].

Discussion:

Currently, the following deduction guide of std::mdspan takes the data handle by reference:

```
template<class MappingType, class AccessorType>
  mdspan(const typename AccessorType::data_handle_type&, const MappingType&,
         const AccessorType&)
    -> mdspan<typename AccessorType::element_type, typename MappingType::extents_type,
              typename MappingType::layout_type, AccessorType>;
```

But the corresponding constructor takes the data handle by value:

```
constexpr mdspan(data_handle_type p, const mapping_type& m, const accessor_type& a);
```

The distinction is observable with `volatile` glvalues. E.g., in the following example, CTAD fails but explicitly specifying template arguments works (demo):

```
#include <cstddef>
#include <mdspan>

int main() {
  int a[1]{};
  int * volatile p = a;
  std::mdspan(
```

```
    p,
    std::layout_right::mapping<std::extents<std::size_t, 1>>{},
    std::default_accessor<int>{}); // error (but accidentally accepted by libc++)
  std::mdspan<int, std::extents<std::size_t, 1>>(
    p,
    std::layout_left::mapping<std::extents<std::size_t, 1>>{},
    std::default_accessor<int>{}); // OK
}
```

Given we're generally passing data handle by value, it seems better to remove `const &` from `const typename AccessorType::data_handle_type&` in the deduction guide, which is more consistent with the constructor and accept more seemingly valid uses.

Note that libc++ is accidentally doing this now by forgetting adding the `&`, see llvm/llvm-project#175024.

[2026-02-20; LWG telecon; Status changed: New → Ready.]

Proposed resolution:

This wording is relative to N5032.

1. Modify 23.7.3.6.1 [mdspan.mdspan.overview], class template `mdspan` synopsis, as indicated:

```
namespace std {
  […]
  template<class MappingType, class AccessorType>
  mdspan(const typename AccessorType::data_handle_type&, const MappingType&,
         const AccessorType&)
    -> mdspan<typename AccessorType::element_type, typename MappingType::extents_type,
              typename MappingType::layout_type, AccessorType>;
}
```

4512. The `system_encoded_string()` and `generic_system_encoded_string()` member functions of `std::filesystem::path` are misnamed

Section: 31.12.6.1 [fs.class.path.general], 31.12.6.3.2 [fs.path.type.cvt], 31.12.6.5.6 [fs.path.native.obs], 31.12.6.5.7 [fs.path.generic.obs], D.22.2 [depr.fs.path.obs] **Status:** Ready **Submitter:** Tom Honermann **Opened:** 2026-01-11 **Last modified:** 2026-02-20

Priority: Not Prioritized

Discussion:

Addresses US 189-304

The `system_encoded_string()` and `generic_system_encoded_string()` member functions of `std::filesystem::path` are misnamed. These functions were added as renamed versions of `string()` and `generic_string()` respectively by P2319R5 ("Prevent path presentation problems"). The original function names are now deprecated.

The C++ standard does not define "system encoding" and casual use of such a term is at best ambiguous. In common language, one might expect the "system encoding" to correspond to the environment encoding (for which `std::text_encoding::environment()` provides a definition) or the encoding of the current locale. However, neither of these encodings corresponds to the encoding these functions are expected to use. 31.12.6.3.2 [fs.path.type.cvt] defines "native encoding" and, for `char`-based filesystem paths, states (in a note) that "For Windows-based operating systems, the native ordinary encoding is determined by calling a Windows API function." It is not specified which Windows API function is consulted, but existing implementations call `AreFileApisANSI()` to choose between the Active Code Page (`CP_ACP`) and the OEM Code Page (`CP_OEMCP`). Microsoft's implementation also checks for a thread-local locale and, if the locale encoding is UTF-8, prefers that (`CP_UTF8`) over either of the other two.

[2026-02-20; LWG telecon; Status changed: New → Ready.]

Proposed resolution:

This wording is relative to N5032.

1. Modify 31.12.6.1 [fs.class.path.general], class `path` synopsis, as indicated:

```
namespace std::filesystem {
  class path {
  public:
    […]
    // 31.12.6.5.6 [fs.path.native.obs], native format observers
    […]
    std::string display_string() const;
    std::string systemnative_encoded_string() const;
    std::wstring wstring() const;
    […]

    // 31.12.6.5.7 [fs.path.generic.obs], generic format observers
    […]
    std::string generic_display_string() const;
    std::string generic_systemnative_encoded_string() const;
    std::wstring generic_wstring() const;
    […]
  };
}
```

2. Modify 31.12.6.3.2 [fs.path.type.cvt] as indicated:

> -2- For member function arguments that take character sequences representing paths and for member functions returning strings, value type and encoding conversion is performed if the value type of the argument or return value differs from `path::value_type`. For the argument or return value, the method of conversion and the encoding to be converted to is determined by its value type:
>
> (2.1) — `char`: The encoding is the native ordinary encoding. The method of conversion, if any, is operating system dependent.
>
> [*Note 1*: For POSIX-based operating systems `path::value_type` is `char` so no conversion from `char` value type arguments or to `char` value type return values is performed. For Windows-based operating systems, the native ordinary encoding is determined by ~~calling a Windows API function~~the current locale encoding and the Windows `AreFileApisANSI` function. — end note]
>
> […]
>
> (2.2) — […]
>
> […]

3. Modify 31.12.6.5.6 [fs.path.native.obs] as indicated:

```
std::string ~~system~~native_encoded_string() const;
std::wstring wstring() const;
std::u8string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;
```

> -7- *Returns*: `native()`.
>
> -8- *Remarks*: Conversion, if any, is performed as specified by 31.12.6.3 [fs.path.cvt].

4. Modify 31.12.6.5.7 [fs.path.generic.obs] as indicated:

```
std::string generic_~~system~~native_encoded_string() const;
std::wstring generic_wstring() const;
std::u8string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;
```

> -5- *Returns*: The pathname in the generic format.
>
> -6- *Remarks*: Conversion, if any, is specified by 31.12.6.3 [fs.path.cvt].

5. Modify D.22.2 [depr.fs.path.obs] as indicated:

```
std::string string() const;
```

> -2- *Returns*: ~~system~~native_encoded_string().

```
std::string generic_string() const;
```

> -3- *Returns*: `generic_`~~system~~native_encoded_string().

4535. Disallow user specialization of `<simd>` templates

Section: 29.10.1 [simd.general] **Status:** Ready **Submitter:** Tim Song **Opened:** 2026-03-05 **Last modified:** 2026-03-06

Priority: Not Prioritized

Discussion:

Addresses US 176-280.

It does not appear that any of the class templates in `<simd>` can be profitably specialized by the user – or that the non-member functions can be used with such user specializations.

[2026-03-06 LWG telecon; move to Ready]

Proposed resolution:

This wording is relative to N5032.

1. Add the following paragraph to the end of 29.10.1 [simd.general]:

> -?- If a program declares an explicit or partial specialization of any of the templates specified in subclause 29.10 [simd], the program is ill-formed, no diagnostic required.

2. Strike 29.10.4 [simd.traits] p3 as redundant:

> ~~-3- The behavior of a program that adds specializations for `alignment` is undefined.~~

4536. Type traits have inconsistent interactions with immediate functions

Section: 21.3.6.4 [meta.unary.prop], 21.3.8 [meta.rel] **Status:** Ready **Submitter:** Tim Song **Opened:** 2026-03-05 **Last modified:** 2026-03-06

Priority: Not Prioritized

View other active issues in [meta.unary.prop].

View all other issues in [meta.unary.prop].

Discussion:

Per 7.7 [expr.const]p23, calls to immediate functions are not immediate invocations, and therefore are not required to result in constant expressions, if the call occurs in an unevaluated operand. Currently, some type traits (e.g. `is_assignable`, `is_invocable`) specify that the construct at issue is treated as an unevaluated operand, while others (`is_constructible`, `is_convertible`) do not. This seems like a bad state of affairs.

The wording below makes everything unevaluated (and hence not check for constant-ness) to match the behavior of `requires` expressions (and implementations). This arguably makes the traits less useful in some contexts where false negatives are tolerable but false positives are not.

[2026-03-06 LWG telecon; move to Ready]

Proposed resolution:

This wording is relative to N5032.

1. Modify [tab:meta.unary.prop], Table 54 — Type property predicates as indicated:

Template	Condition	Preconditions
…	…	…
`template<class T, class U>` `struct reference_constructs_from_temporary;`	`T` is a reference type, and the initialization `T t(`*`VAL`*`<U>);` is well-formed and binds `t` to a temporary object whose lifetime is extended (6.8.7 [class.temporary]). The full-expression of the variable initialization is treated as an unevaluated operand (7.2.3 [expr.context]). Access checking is performed as if in a context unrelated to `T` and `U`. Only the validity of the immediate context of the variable initialization is considered. [*Note 5: … — end note*]	[…]
`template<class T, class U>` `struct reference_converts_from_temporary;`	`T` is a reference type, and the initialization `T t =` *`VAL`*`<U>;` is well-formed and binds `t` to a temporary object whose lifetime is extended (6.8.7 [class.temporary]). The full-expression of the variable initialization is treated as an unevaluated operand (7.2.3 [expr.context]). Access checking is performed as if in a context unrelated to `T` and `U`. Only the validity of the immediate context of the variable initialization is considered. [*Note 6: … — end note*]	[…]
…	…	…

2. Modify 21.3.6.4 [meta.unary.prop] p9 as indicated:

> -9- The predicate condition for a template specialization `is_constructible<T, Args...>` shall be satisfied if and only if the following variable definition would be well-formed for some invented variable `t`:
>
> ```
> T t(declval<Args>()...);
> ```
>
> [*Note 7: … — end note*]
>
> The full-expression of the variable initialization is treated as an unevaluated operand (7.2.3 [expr.context]). Access checking is performed as if in a context unrelated to `T` and any of the `Args`. Only the validity of the immediate context of the variable initialization is considered.
>
> [*Note 8: … — end note*]

3. Modify 21.3.8 [meta.rel] p6 as indicated:

> -9- The predicate condition for a template specialization `is_convertible<From, To>` shall be satisfied if and only if the return ~~expression~~statement (8.8.4 [stmt.return]) in the following code would be well-formed, including any implicit conversions to the return type of the function:
>
> ```
> To test() {
>   return declval<From>();
> }
> ```
>
> [*Note 4: … — end note*]
>
> Access checking is performed in a context unrelated to `To` and `From`. The operand of the `return` statement (including initialization of the returned object or reference, if any) is treated as an unevaluated operand (7.2.3 [expr.context]), and only the validity of its immediate context is considered. ~~Only the validity of the immediate context of the *expression* of the `return` statement (8.8.4 [stmt.return]) (including initialization of the returned object or reference) is considered.~~
>
> [*Note 5: … — end note*]

Tentatively Ready Issues

3831. Two-digit formatting of negative `year` is ambiguous

Section: 30.12 [time.format], 30.13 [time.parse] **Status:** Tentatively Ready **Submitter:** Matt Stephanson **Opened:** 2022-11-18 **Last modified:** 2025-12-04

Priority: 3

View other active issues in [time.format].

View all other issues in [time.format].

Discussion:

An issue has been identified regarding the two-digit formatting of negative years according to Table [tab:time.format.spec] (30.12 [time.format]):

```
cout << format("{:%y} ", 1976y)  // "76"
     << format("{:%y}", -1976y); // also "76"?
```

The relevant wording is

> The last two decimal digits of the year. If the result is a single digit it is prefixed by `0`. The modified command `%Oy` produces the locale's alternative representation. The modified command `%Ey` produces the locale's alternative representation of offset from `%EC` (year only).

MSVC STL treats the regular modified form symmetrically. Just as `%Ey` is the offset from `%EC`, so `%y` is the offset from `%C`, which is itself "[t]he year divided by 100 using *floored division*." (emphasis added). Because -1976 is the 24th year of the -20th century, the above code will print "76 24" using MSVC STL. However, many users expect, and libc++ gives, a result based on the literal wording, "76 76".

IEEE 1003.1-2008 strftime expects the century to be nonnegative, but the glibc implementation prints 24 for -1976. My own opinion is that this is the better result, because it consistently interprets `%C` and `%y` as the quotient and remainder of floored division by 100.

Howard Hinnant, coauthor of the original 30.12 [time.format] wording in P0355 adds:

> On the motivation for this design it is important to remember a few things:
>
> - POSIX `strftime`/`strptime` doesn't handle negative years in this department, so this is an opportunity for an extension in functionality.
> - This is a formatting/parsing issue, as opposed to a computational issue. This means that human readability of the string syntax is the most important aspect. Computational simplicity takes a back seat (within reason).
> - `%C` can't be truncated division, otherwise the years [-99, -1] would map to the same century as the years [0, 99]. So floored division is a pretty easy and obvious solution.
> - `%y` is obvious for non-negative years: The last two decimal digits, or `y % 100`.
>
> This leaves how to represent negative years with `%y`. I can think of 3 options:
>
> 1. Use the last two digits without negating: -1976 → 76.
> 2. Use the last two digits and negate it: -1976 → -76.
> 3. Use floored modulus arithmetic: -1976 → 24.
>
> The algorithm to convert `%C` and `%y` into a year is not important to the client because these are both strings, not integers. The client will do it with `parse`, not `100*C + y`.
>
> I discounted solution 3 as not sufficiently obvious. If the output for -1976 was 23, the human reader wouldn't immediately know that this is off by 1. The reader is expecting the POSIX spec:
>
> > the last two digits of the year as a decimal number [00,99].
>
> 24 just doesn't cut it.
>
> That leaves solution 1 or 2. I discounted solution 2 because having the negative in 2 places (the `%C` and `%y`) seemed overly complicated and more error prone. The negative sign need only be in one place, and it has to be in `%C` to prevent ambiguity.
>
> That leaves solution 1. I believe this is the solution for an extension of the POSIX spec to negative years with the property of least surprise to the client. The only surprise is in `%C`, not `%y`, and the surprise in `%C` seems unavoidable.

[2022-11-30; Reflector poll]

Set priority to 3 after reflector poll.

A few votes for priority 2. Might need to go to LEWG.

Previous resolution [SUPERSEDED]:

This wording is relative to N4917.

[*Drafting Note:* Two mutually exclusive options are prepared, depicted below by **Option A** and **Option B**, respectively.]

Option A: This is Howard Hinnant's choice (3)

1. Modify 30.12 [time.format], Table [tab:time.format.spec] as indicated:

Table 102 — Meaning of conversion specifiers [tab:time.format.spec]

Specifier	Replacement
	[…]
%y	The ~~last two decimal digits of the year~~remainder after dividing the year by 100 using floored division. If the result is a single digit it is prefixed by `0`. The modified command `%Oy` produces the locale's alternative representation. The modified command `%Ey` produces the locale's alternative representation of offset from `%EC` (year only).
	[…]

2. Modify 30.13 [time.parse], Table [tab:time.parse.spec] as indicated:

Table 103 — Meaning of `parse` flags [tab:time.parse.spec]

Flag	Parsed value
	[…]
%y	The ~~last two decimal digits of the year~~remainder after dividing the year by 100 using floored division. If the century is not otherwise specified (e.g. with `%C`), values in the range `[69, 99]` are presumed to refer to the years 1969 to 1999, and values in the range `[00, 68]` are presumed to refer to the years 2000 to 2068. The modified command `%N y` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified commands `%Ey` and `%Oy` interpret the locale's alternative representation.
	[…]

Option B: This is Howard Hinnant's choice (1)

1. Modify 30.12 [time.format], Table [tab:time.format.spec] as indicated:

Table 102 — Meaning of conversion specifiers [tab:time.format.spec]

Specifier	Replacement
	[…]
%y	The last two decimal digits of the year, regardless of the sign of the year. If the result is a single digit it is prefixed by `0`. The modified command `%Oy` produces the locale's alternative representation. The modified command `%Ey` produces the locale's alternative representation of offset from `%EC` (year only). [*Example ?:* `cout << format("{:%C %y}", -1976y);` prints `-20 76`. — *end example*]
	[…]

2. Modify 30.13 [time.parse], Table [tab:time.parse.spec] as indicated:

Table 103 — Meaning of `parse` flags [tab:time.parse.spec]

Flag	Parsed value
	[…]
%y	The last two decimal digits of the year, regardless of the sign of the year. If the century is not otherwise specified (e.g. with `%C`), values in the range `[69, 99]` are presumed to refer to the years 1969 to 1999, and values in the range `[00, 68]` are presumed to refer to the years 2000 to 2068. The modified command `%N y` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified commands `%Ey` and `%Oy` interpret the locale's alternative representation. [*Example ?:* `year y; istringstream{"-20 76"} >> parse("%3C %y", y);` results in `y == -1976y`. — *end example*]
	[…]

[2025-10-17; Jonathan provides updated wording using Option B]

[2025-12-04; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 30.12 [time.format], Table [tab:time.format.spec] as indicated:

Table 133 — Meaning of conversion specifiers [tab:time.format.spec]

Specifier	Replacement
	[…]
%y	The last two decimal digits of the year, regardless of the sign of the year. If the result is a single digit it is prefixed by `0`. The modified command `%Oy` produces the locale's alternative representation. The modified command `%Ey` produces the locale's alternative representation of offset from `%EC` (year only). [*Example ?:* `cout << format("{:%C %y}", -1976y);` prints `-20 76`. — *end example*]
	[…]

2. Modify 30.13 [time.parse], Table [tab:time.parse.spec] as indicated:

Table 103 — Meaning of `parse` flags [tab:time.parse.spec]

Flag	Parsed value
	[…]
%y	The last two decimal digits of the year, regardless of the sign of the year. If the century is not otherwise specified (e.g. with `%C`), values in the range [`69`, `99`] are presumed to refer to the years 1969 to 1999, and values in the range [`00`, `68`] are presumed to refer to the years 2000 to 2068. The modified command `%N y` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified commands `%Ey` and `%Oy` interpret the locale's alternative representation. [*Example ?:* `year y; istringstream{"-20 76"} >> parse("%3C %y", y);` results in `y == -1976y`. — *end example*]
	[…]

4090. Underspecified use of locale facets for locale-dependent `std::format`

Section: 28.5.2.2 [format.string.std] **Status:** Tentatively Ready **Submitter:** Jens Maurer **Opened:** 2024-04-30 **Last modified:** 2026-01-19

Priority: 3

View other active issues in [format.string.std].

View all other issues in [format.string.std].

Discussion:

There are `std::format` variants that take an explicit `std::locale` parameter. There is the `"L"` format specifier that uses that locale (or some environment locale) for formatting, according to 28.5.2.2 [format.string.std] p17:

> "For integral types, the locale-specific form causes the context's locale to be used to insert the appropriate digit group separator characters."

It is unclear which specific facets are used to make this happen. This is important, because users can install their own facets into a given locale. Specific questions include:

- Is `num_put<>` being used? Or just `numpunct<>`?
- Are any of the `_byname` facets being used?

Assuming the encoding for `char` is UTF-8, the use of a user-provided `num_put<>` facet (as opposed to `std::format` creating the output based on `numpunct<>`) would allow digit separators that are not expressibly as a single UTF-8 code unit.

[2024-05-08; Reflector poll]

Set priority to 3 after reflector poll.

[2024-06-12; SG16 meeting]

The three major implementations all use `numpunct` but not `num_put`, clarify that this is the intended behaviour.

[2025-06-12; Jonathan provides wording]

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

1. Modify 28.5.2.2 [format.string.std] as indicated:

-17- When the `L` option is used, the form used for the conversion is called the *locale-specific* form. The `L` option is only valid for arithmetic types, and its effect depends upon the type.

(17.1) — For integral types, the locale-specific form causes the context's locale to be used to insert the appropriate digit group separator characters as if obtained with `numpunct<charT>::grouping` and `numpunct<charT>::thousands_sep`.

(17.2) — For floating-point types, the locale-specific form causes the context's locale to be used to insert the appropriate digit group and radix separator characters as if obtained with `numpunct<charT>::grouping`, `numpunct<charT>::thousands_sep`, and `numpunct<charT>::decimal_point`.

(17.3) — For the textual representation of `bool`, the locale-specific form causes the context's locale to be used to insert the appropriate string as if obtained with `numpunct<charT>::truename` or `numpunct<charT>::falsename`.

[2025-08-27; SG16 meeting]

SG16 unanimously approved new wording from Victor. The new wording incorporates similar wording as added by P2419R2 to address 3565(i). Status updated SG16 → Open.

[2026-01-16; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 28.5.2.2 [format.string.std] as indicated:

-17- When the `L` option is used, the form used for the conversion is called the *locale-specific* form. The `L` option is only valid for arithmetic types, and its effect depends upon the type.

(17.1) — For integral types, the locale-specific form causes the context's locale to be used to insert the appropriate digit group separator characters as if obtained with `numpunct<charT>::grouping` and `numpunct<charT>::thousands_sep`.

(17.2) — For floating-point types, the locale-specific form causes the context's locale to be used to insert the appropriate digit group and radix separator characters as if obtained with `numpunct<charT>::grouping`, `numpunct<charT>::thousands_sep`, and `numpunct<charT>::decimal_point`.

(17.3) — For the textual representation of `bool`, the locale-specific form causes the context's locale to be used to insert the appropriate string as if obtained with `numpunct<charT>::truename` or `numpunct<charT>::falsename`.

If the string literal encoding is a Unicode encoding form and the locale is among an implementation-defined set of locales, each replacement that depends on the locale is performed as if the replacement character sequence is converted to the string literal encoding.

4130. Preconditions for `std::launder` might be overly strict

Section: 17.6.5 [ptr.launder] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2024-07-30 **Last modified:** 2026-02-13

Priority: 3

View all other issues in [ptr.launder].

Discussion:

From issue cplusplus/draft#4553 which is considered non-editorial.

Currently, `std::launder` has a precondition that the pointed to object must be within its lifetime. However, per the example added by P0593R6, it's probably intended that the result of `std::launder` should be allowed to point to an array element subobject whose lifetime has not started yet.

[2024-10-02; Reflector poll]

Set priority to 3 after reflector poll. Needs review by Core.

[Wrocław 2024-11-18; approved by Core]

[2026-02-13; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N4986.

1. Modify 17.6.5 [ptr.launder] as indicated:

```
template<class T> [[nodiscard]] constexpr T* launder(T* p) noexcept;
```

-1- *Mandates*: `!is_function_v<T> && !is_void_v<T>` is `true`.

-2- *Preconditions*: `p` represents the address *A* of a byte in memory. An object *X* ~~that is within its lifetime (6.8.4 [basic.life]) and~~ whose type is similar (7.3.6 [conv.qual]) to `T` is located at the address *A*, and is either within its lifetime (6.8.4 [basic.life]) or is an array element subobject whose containing array object is within its lifetime. All bytes of storage that would be reachable through (6.9.4 [basic.compound]) the result are reachable through `p`.

-3- *Returns*: A value of type `T*` that points to *X*.

[…]

4259. P1148R0 changed the return values of searching functions of `std::basic_string` on some platforms

Section: 27.4.3.8.2 [string.find] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2025-05-05 **Last modified:** 2025-12-04

Priority: 3

View other active issues in [string.find].

View all other issues in [string.find].

Discussion:

P1148R0 respecified the searching functions of `std::basic_string` to return corresponding string view type's `npos` member constant (equal to `std::size_t(-1)`), converted to the string type `S`'s member `S::size_type`, when the search fails. Before the change, `S::npos` (equal to `S::size_type(-1)`) was returned on failure.

On platforms where `std::size_t` isn't the widest unsigned integer type (e.g. on usual 32-bit platforms), the return value can change. Because there can be an allocator with a wider size_type, and when the basic_string type `S` uses such an allocator, `S::size_type` is specified to be that type, which in turn makes `S::size_type(std::size_t(-1))` not equal to `S::size_type(-1)`.

Do we want to restore the old return values?

Previous resolution [SUPERSEDED]:

This wording is relative to N5008.

1. Modify 27.4.3.8.2 [string.find] as indicated:

```
template<class T>
  constexpr size_type find(const T& t, size_type pos = 0) const noexcept(see below);
[…]
template<class T>
  constexpr size_type find_last_not_of(const T& t, size_type pos = npos) const noexcept(see below);
```

-2- *Constraints*: […]

-3- *Effects*: Let *G* be the name of the function. Equivalent to:

```
basic_string_view<charT, traits> s = *this, sv = t;
return s.G(sv, pos);
if (auto result = s.G(sv, pos); result == size_t(-1))
  return npos;
else
  return result;
```

[2025-06-10, reflector discussion]

During reflector discussion of this issue there was a preference to adjust the proposed wording to use `s.npos` instead of `size_t(-1)`.

[2025-10-21; Reflector poll.]

Set priority to 3 after reflector poll.

[2025-12-04; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5008.

1. Modify 27.4.3.8.2 [string.find] as indicated:

```
template<class T>
  constexpr size_type find(const T& t, size_type pos = 0) const noexcept(see below);
[…]
template<class T>
  constexpr size_type find_last_not_of(const T& t, size_type pos = npos) const noexcept(see below);
```

-2- *Constraints*: […]

-3- *Effects*: Let *G* be the name of the function. Equivalent to:

```
basic_string_view<charT, traits> s = *this, sv = t;
return s.G(sv, pos);
```

```
if (auto result = s.G(sv, pos); result == s.npos)
  return npos;
else
  return result;
```

4324. `unique_ptr<void>::operator*` is not SFINAE-friendly

Section: 20.3.1.3.5 [unique.ptr.single.observers] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2025-08-24 **Last modified:** 2025-12-04

Priority: 3

View other active issues in [unique.ptr.single.observers].

View all other issues in [unique.ptr.single.observers].

Discussion:

LWG 2762[(i)] added a conditional `noexcept` specification to `unique_ptr::operator*` to make it consistent with `shared_ptr::operator*`:

```
constexpr add_lvalue_reference_t<T> operator*() const noexcept(noexcept(*declval<pointer>()));
```

This unexpectedly makes `unique_ptr<void>::operator*` no longer SFINAE-friendly, for example:

```
#include <memory>

template<class T> concept dereferenceable = requires(T& t) { *t; };

static_assert( dereferenceable<int *>);
static_assert(!dereferenceable<void*>);

static_assert( dereferenceable<std::shared_ptr<int >>);
static_assert(!dereferenceable<std::shared_ptr<void>>);

static_assert( dereferenceable<std::unique_ptr<int >>);
static_assert( dereferenceable<std::unique_ptr<void>>); // hard error
```

Given that the standard intends for `operator*` of `shared_ptr` and `unique_ptr` to be SFINAE-friendly based on 20.3.2.2.6 [util.smartptr.shared.obs], regardless of the value of `static_assert`, it is reasonable to assume that there should be no hard error here.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 20.3.1.3.5 [unique.ptr.single.observers] as indicated:

```
constexpr add_lvalue_reference_t<T> operator*() const noexcept(~~noexcept(*declval<pointer>())~~see below);
```

-1- *Mandates*: `reference_converts_from_temporary_v<add_lvalue_reference_t<T>, decltype(*declval<pointer>())>` is `false`.

-2- *Preconditions*: `get() != nullptr` is `true`.

-3- *Returns*: `*get()`.

-?- *Remarks*:: The exception specification is equivalent to:

```
!requires { *declval<pointer>(); } || requires { { *declval<pointer>() } noexcept; }
```

[2025-08-26; Reflector discussion]

During reflector triaging it had been pointed out that a better solution would be to constrain the `operator*` directly. The proposed wording has been updated to that effect.

[2025-10-22; Reflector poll.]

Set priority to 3 after reflector poll.

[2025-12-04; Reflector poll.]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 20.3.1.3.5 [unique.ptr.single.observers] as indicated:

```
constexpr add_lvalue_reference_t<T> operator*() const noexcept(noexcept(*declval<pointer>()));
```

-?- *Constraints*: `*declval<pointer>()` is a well-formed expression.

-1- *Mandates*: `reference_converts_from_temporary_v<add_lvalue_reference_t<T>, decltype(*declval<pointer>())>` is `false`.

-2- *Preconditions*: `get() != nullptr` is `true`.

-3- *Returns*: `*get()`.

4378. Inconsistency between `std::basic_string`'s `data()` and `operator[]` specification

Section: 27.4.3.6 [string.access] **Status:** Tentatively Ready **Submitter:** Peter Bindels **Opened:** 2025-09-16 **Last modified:** 2026-03-06

Priority: 4

View all other issues in [string.access].

Discussion:

From the working draft N5014, the specification for `operator[]` in 27.4.3.6 [string.access] p2 says:

> *Returns*: `*(begin() + pos)` if `pos < size()`. Otherwise, returns a reference to an object of type `charT` with value `charT()`, where modifying the object to any value other than `charT()` leads to undefined behavior.

The specification for data() in 27.4.3.8.1 [string.accessors] p1 (and p4) says, however:

> *Returns*: A pointer `p` such that `p + i == addressof(operator[](i))` for each `i` in `[0, size()]`.

The former implies that `str[str.size()]` is allowed to be the address of any null terminator, while the latter restricts it to only being the null terminator belonging to the string.

Suggested fix: Change wording around `operator[]` to

> *Returns*: `*(begin() + pos)` if `pos <= size()`. The program shall not modify the value stored at `size()` to any value other than `charT()`; otherwise, the behavior is undefined.

This moves it inline with the `data()` specification. Given the hardened precondition that `pos <= size()` this does not change behavior for any in-contract access, and we do not define what the feature does when called with broken preconditions. I have been looking at the latter but that will be an EWG paper instead.

[2025-10-21; Reflector poll.]

Set priority to 4 after reflector poll.

"NAD. `begin() + size()` is not dereferenceable and should remain that way."

"Saying "if `pos <= size()` is redundant given the precondition above."

"The resolution removes any guarantee that the value at `str[str.size()]` is `charT()`. Furthermore, the premise of the issue is incorrect, returning the address of a different null terminator not belonging to the string would make traversing it with other string operations UB, so it has to return a reference to a terminator that's within the same array."

"`*(begin() = size())` is UB, but could use `*(data() + size())` instead. Personally I'd like `*end()` to be valid, but that's certainly LEWG business requiring a paper."

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 27.4.3.6 [string.access] as indicated:

```
constexpr const_reference operator[](size_type pos) const;
constexpr       reference operator[](size_type pos);
```

-1- *Hardened preconditions*: `pos <= size()` is `true`.

-2- *Returns*: `*(begin() + pos)` if `pos <= size()`. ~~Otherwise, returns a reference to an object of type `charT` with value `charT()`, where modifying the object to any value other than `charT()` leads to undefined behavior.~~

-3- *Throws*: Nothing.

-4- *Complexity*: Constant time.

-?- *Remarks* The program shall not modify the value stored at `size()` to any value other than `charT()`; otherwise, the behavior is undefined

[2025-11-11; Jonathan provides new wording]

We say that `basic_string` is a contiguous container, which makes the `addressof` wording in `c_str()` and `data()` redundant. The front matter says that there's a null terminator present, so we can move the rule about not modifying the terminator there instead of repeating it in `operator[]` and `c_str()`.

We can also permit modifying the string contents through `const_cast<char*>(str.c_str())[0]`. There's no reason for that to be undefined when `const_cast<string&>(str)[0]` and `const_cast<string&>(str).data()[0]` are both allowed. The only restriction should be on changing the null terminator. Changing any other characters through `c_str() const` or `data() const` is no different to changing them through the non-const `data()`, and does not need to cause undefined behaviour.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 27.4.3.1 [basic.string.general] as indicated:

   -3- In all cases, `[data(), data() + size()]` is a valid range, `data() + size()` points at an object with value `charT()` (a "null terminator"), and `size() <= capacity()` is `true`. Non-const access to the null terminator is possible, e.g. using `*(data()+size())`, but the program has undefined behavior if the null terminator is modified to any value other than `charT()`.

2. Modify 27.4.3.6 [string.access] as indicated:

```
constexpr const_reference operator[](size_type pos) const;
constexpr reference       operator[](size_type pos);
```

   -1- *Hardened Preconditions*: `pos <= size()` is `true`.

   -2- *Returns*: `*(data() + pos)`. ~~`*(begin() + pos)` if `pos < size()`. Otherwise, returns a reference to an object of type `charT` with value `charT()`, where modifying the object to any value other than `charT()` leads to undefined behavior.~~

   -3- *Throws*: Nothing.

   -4- *Complexity*: Constant time.

3. Modify 27.4.3.8.1 [string.accessors] as indicated:

```
constexpr const charT* c_str() const noexcept;
constexpr const charT* data() const noexcept;
constexpr charT* data() noexcept;
```

   -1- *Returns*: `to_address(begin())`. ~~A pointer `p` such that `p + i == addressof(operator[](i))` for each `i` in `[0,size()]`.~~

   -2- *Complexity*: Constant time.

   ~~-3- *Remarks*: The program shall not modify any of the values stored in the character array; otherwise, the behavior is undefined.~~

~~`constexpr charT* data() noexcept;`~~

   ~~-4- *Returns*: A pointer `p` such that `p + i == addressof(operator[](i))` for each `i` in `[0,size()]`.~~

   ~~-5- *Complexity*: Constant time.~~

   ~~-6- *Remarks*: The program shall not modify the value stored at `p + size()` to any value other than `charT()`; otherwise, the behavior is undefined.~~

[2026-02-27; Jonathan provides new wording]

[2026-03-06; Reflector poll.]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Modify 27.4.3.1 [basic.string.general] as indicated:

   -3- In all cases, `[data(), data() + size()]` is a valid range, `data() + size()` points at an object with value `charT()` (a "null terminator"), and `size() <= capacity()` is `true`. The program has undefined behavior if the null terminator is modified to any value other than `charT()`.

2. Modify 27.4.3.6 [string.access] as indicated:

```
constexpr const_reference operator[](size_type pos) const;
constexpr reference       operator[](size_type pos);
```

   -1- *Hardened Preconditions*: `pos <= size()` is `true`.

-2- *Returns*: `data()[pos].` ~~`*(begin() + pos) if pos < size(). Otherwise, returns a reference to an object of type charT with value charT(), where modifying the object to any value other than charT() leads to undefined behavior.`~~

-3- *Throws*: Nothing.

-4- *Complexity*: Constant time.

3. Modify 27.4.3.8.1 [string.accessors] as indicated:

```
constexpr const charT* c_str() const noexcept;
constexpr const charT* data() const noexcept;
constexpr charT* data() noexcept;
```

-1- *Returns*: `to_address(begin()).` ~~`A pointer p such that p + i == addressof(operator[](i)) for each i in [0,size()].`~~

-2- *Complexity*: Constant time.

~~*-3- Remarks*: The program shall not modify any of the values stored in the character array; otherwise, the behavior is undefined.~~

~~`constexpr charT* data() noexcept;`~~

~~*-4- Returns*: A pointer p such that `p + i == addressof(operator[](i))` for each i in `[0,size()]`.~~

~~*-5- Complexity*: Constant time.~~

~~*-6- Remarks*: The program shall not modify the value stored at `p + size()` to any value other than `charT()`; otherwise, the behavior is undefined.~~

4457. freestanding for `stable_sort`, `stable_partition` and `inplace_merge`

Section: 26.4 [algorithm.syn] **Status:** Tentatively Ready **Submitter:** Braden Ganetsky **Opened:** 2025-11-06 **Last modified:** 2026-01-16

Priority: Not Prioritized

View all other issues in [algorithm.syn].

Discussion:

Addresses US 157-255

This applies the resolution for US 157-255.

[2026-01-16; Reflector poll.]

Set status to Tentatively Ready after ten votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 26.4 [algorithm.syn], as indicated:

```
namespace ranges {
  template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
           class Proj = identity>
    requires sortable<I, Comp, Proj>
    constexpr I stable_sort(I first, S last, Comp comp = {}, Proj proj = {});         // hosted
  template<random_access_range R, class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    constexpr borrowed_iterator_t<R>
      stable_sort(R&& r, Comp comp = {}, Proj proj = {});                             // hosted

  template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
           class Comp = ranges::less, class Proj = identity>
    requires sortable<I, Comp, Proj>
    I stable_sort(Ep&& exec, I first, S last, Comp comp = {},
                  Proj proj = {});                                        // freestanding-deletedhosted
  template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
           class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    borrowed_iterator_t<R>
      stable_sort(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});      // freestanding-deletedhosted
}
```

2. Modify 26.4 [algorithm.syn], as indicated:

```
namespace ranges {
  template<bidirectional_iterator I, sentinel_for<I> S, class Proj = identity,
           indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    constexpr subrange<I> stable_partition(I first, S last, Pred pred,            // hosted
                                           Proj proj = {});
  template<bidirectional_range R, class Proj = identity,
```

```
          indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    constexpr borrowed_subrange_t<R> stable_partition(R&& r, Pred pred,          // hosted
                                                      Proj proj = {});

  template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
           class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    subrange<I>
      stable_partition(Ep&& exec, I first, S last, Pred pred,
                       Proj proj = {});                                  // ~~freestanding-deleted~~hosted
  template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
           indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R>
      stable_partition(Ep&& exec, R&& r, Pred pred, Proj proj = {});     // ~~freestanding-deleted~~hosted
}
```

3. Modify 26.4 [algorithm.syn], as indicated:

```
namespace ranges {
  template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
           class Proj = identity>
    requires sortable<I, Comp, Proj>
    constexpr I
      inplace_merge(I first, I middle, S last, Comp comp = {}, Proj proj = {});       // hosted
  template<bidirectional_range R, class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    constexpr borrowed_iterator_t<R>
      inplace_merge(R&& r, iterator_t<R> middle, Comp comp = {}, Proj proj = {});     // hosted

  template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
           class Comp = ranges::less, class Proj = identity>
    requires sortable<I, Comp, Proj>
    I inplace_merge(Ep&& exec, I first, I middle, S last, Comp comp = {},
                    Proj proj = {});                          // ~~freestanding-deleted~~hosted
  template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
           class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    borrowed_iterator_t<R>
      inplace_merge(Ep&& exec, R&& r, iterator_t<R> middle, Comp comp = {},
                    Proj proj = {});                          // ~~freestanding-deleted~~hosted
}
```

4460. Missing *Throws*: for last `variant` constructor

Section: 22.6.3.2 [variant.ctor] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2025-11-07 **Last modified:** 2025-12-04

Priority: Not Prioritized

View other active issues in [variant.ctor].

View all other issues in [variant.ctor].

Discussion:

All `variant` constructors except the last one have a *Throws*: element saying what they're allowed to throw.

This originates from an editorial pull request, where the submitter said:

> "It looks like this defect is an artifact of a change between P0088R0 and P0088R1. Note how in R0 neither one of the `emplaced_type_t`/`emplaced_index_t` (as they were then called) + `initializer_list` constructors have a throws clause. In R1 only one of them gained it."

Previous resolution [SUPERSEDED]:

This wording is relative to N5014.

1. Modify 22.6.3.2 [variant.ctor], as indicated:

```
template<size_t I, class U, class... Args>
  constexpr explicit variant(in_place_index_t<I>, initializer_list<U> il, Args&&... args);
```

-35- *Constraints*:

(35.1) — `I` is less than `sizeof...(Types)` and
(35.2) — `is_constructible_v<`T_I`, initializer_list<U>&, Args...>` is `true`.

-36- *Effects*: Direct-non-list-initializes the contained value of type T_I with `il, std::forward<Args>(args)...`.

-37- *Postconditions*: `index()` is `I`.

-?- *Throws*: Any exception thrown by calling the selected constructor of T_I.

-38- *Remarks*: If T_I's selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

[2025-11-11; Jonathan provides improved wording]

[2025-12-04; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 22.6.3.2 [variant.ctor], as indicated:

```
constexpr variant() noexcept(see below);
```

-2- *Constraints*: `is_default_constructible_v<`T_0`>` is `true`.

-3- *Effects*: Constructs a `variant` holding a value-initialized value of type T_0.

-4- *Postconditions*: `valueless_by_exception()` is `false` and `index()` is `0`.

-5- *Throws*: Any exception thrown by the value-initialization of T_0.

-6- *Remarks*: […]

```
constexpr variant(const variant&);
```

-7- *Effects*: If `w` holds a value, initializes the `variant` to hold the same alternative as `w` and direct-initializes the contained value with *GET*`<j>(w)`, where `j` is `w.index()`. Otherwise, initializes the `variant` to not hold a value.

-8- *Throws*: Any exception thrown by ~~direct-initializating any T_i for all i~~ the initialization of the contained value.

-9- *Remarks*: […]

```
constexpr variant(variant&&) noexcept(see below);
```

-10- *Constraints*: `is_move_constructible_v<`T_i`>` is `true` for all i.

-11- *Effects*: If `w` holds a value, initializes the `variant` to hold the same alternative as `w` and direct-initializes the contained value with *GET*`<j>(std::move(w))`, where `j` is `w.index()`. Otherwise, initializes the `variant` to not hold a value.

-12- *Throws*: Any exception thrown by ~~move-constructing any T_i for all i~~ the initialization of the contained value.

-13- *Remarks*: […]

```
template<class T> constexpr variant(T&&) noexcept(see below);
```

-14- Let T_j be a type that is determined as follows: build an imaginary function *FUN*`(`T_i`)` for each alternative type T_i for which T_i `x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x`. The overload *FUN*`(`T_j`)` selected by overload resolution for the expression *FUN*`(std::forward<T>(t))` defines the alternative T_j which is the type of the contained value after construction.

-15- *Constraints*: […]

-16- *Effects*: Initializes `*this` to hold the alternative type T_j and direct-non-list-initializes the contained value with `std::forward<T>(t)`.

-17- *Postconditions*: […]

-18- *Throws*: Any exception thrown by the initialization of the ~~selected alternative T_j~~ contained value.

-19- *Remarks*: […]

```
template<class T, class... Args> constexpr variant(in_place_type_t<T>, Args&&... args);
```

-20- *Constraints*: […]

-21- *Effects*: Direct-non-list-initializes the contained value of type `T` with `std::forward<Args>(args)...`.

-22- *Postconditions*: […]

-23- *Throws*: Any exception thrown by ~~the selected constructor of `T`~~ the initialization of the contained value.

-24- *Remarks*: […]

```
template<class T, class U, class... Args>
  constexpr variant(in_place_type_t<T>, initializer_list<U> li, Args&&... args);
```

-25- *Constraints*: […]

-26- *Effects*: Direct-non-list-initializes the contained value of type `T` with `il, std::forward<Args>(args)...`.

-27- *Postconditions*: […]

-28- *Throws*: Any exception thrown by ~~the selected constructor of `T`~~ the initialization of the contained value.

-29- *Remarks*: […]

```
template<size_t I, class... Args>
  constexpr explicit variant(in_place_index_t<I>, Args&&... args);
```

-30- *Constraints*:

(30.1) — `I` is less than `sizeof...(Types)` and
(30.2) — `is_constructible_v<`T_I`, Args...>` is `true`.

-31- *Effects*: Direct-non-list-initializes the contained value of type T_I with `std::forward<Args>(args)...`.

-32- *Postconditions*: `index()` is `I`.

-33- *Throws*: Any exception thrown by ~~the selected constructor of `T`~~$_I$ the initialization of the contained value.

-34- *Remarks*: If T_I's selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

```
template<size_t I, class U, class... Args>
  constexpr explicit variant(in_place_index_t<I>, initializer_list<U> il, Args&&... args);
```

-35- *Constraints*:

(35.1) — `I` is less than `sizeof...(Types)` and
(35.2) — `is_constructible_v<`T_I`, initializer_list<U>&, Args...>` is `true`.

-36- *Effects*: Direct-non-list-initializes the contained value of type T_I with `il, std::forward<Args>(args)...`.

-37- *Postconditions*: `index()` is `I`.

-?- *Throws*: Any exception thrown by the initialization of the contained value.

-38- *Remarks*: If T_I's selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

4467. `hive::splice` can throw `bad_alloc`

Section: 23.3.9.5 [hive.operations] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2025-11-07 **Last modified:** 2025-12-04

Priority: Not Prioritized

View other active issues in [hive.operations].

View all other issues in [hive.operations].

Discussion:

Moving blocks from the source `hive` to the destination `hive` might require reallocating the array of pointers to blocks, so the *Throws*: element should allow this.

[2025-12-04; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 23.3.9.5 [hive.operations], as indicated:

```
void splice(hive& x);
void splice(hive&& x);
```

-2- *Preconditions*: `get_allocator() == x.get_allocator()` is `true`.

-3- *Effects*: If `addressof(x) == this` is `true`, the behavior is erroneous and there are no effects. Otherwise, inserts the contents of `x` into `*this` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

-4- *Throws*: `length_error` if any of `x`'s active blocks are not within the bounds of *`current-limits`* , as well as any exceptions thrown by the allocator.

-5- *Complexity*: Linear in the sum of all element blocks in `x` plus all element blocks in `*this`.

-6- *Remarks*: Reserved blocks in `x` are not transferred into `*this`. If `addressof(x) == this` is `false`, invalidates the past-the-end iterator for both `x` and `*this`.

4468. §[const.wrap.class] "`operator decltype(auto)`" is ill-formed

Section: 21.3.5 [const.wrap.class] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2025-11-07 **Last modified:** 2025-12-04

Priority: Not Prioritized

View other active issues in [const.wrap.class].

View all other issues in [const.wrap.class].

Discussion:

Following the approval of CWG 1670 in Kona 2025, the following declaration in class template `constant_wrapper`, 21.3.5 [const.wrap.class], is ill-formed:

```
constexpr operator decltype(auto)() const noexcept { return value; }
```

[2025-12-04; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 21.3.5 [const.wrap.class], class template `constant_wrapper` synopsis, as indicated:

```
[…]
template<cw-fixed-value X, class>
struct constant_wrapper : cw-operators {
  static constexpr const auto & value = X.data;

  using type = constant_wrapper;
  using value_type = typename decltype(X)::type;

  […]
  constexpr operator decltype(~~auto~~value)() const noexcept { return value; }
};
```

4474. "`round_to_nearest`" rounding mode is unclear

Section: 17.3.4 [round.style] **Status:** Tentatively Ready **Submitter:** Jan Schultke **Opened:** 2025-11-13 **Last modified:** 2026-01-16

Priority: Not Prioritized

Discussion:

Consider the specification of `round_to_nearest` in 17.3.4 [round.style]:

> `round_to_nearest` if the rounding style is to the nearest representable value

It is unclear how exact ties are rounded. For example, with this rounding, would a value that is equidistant to zero and `numeric_limits<float>::min()` be rounded towards zero or away from zero?

In 17.3.5.2 [numeric.limits.members], there exists a footnote for `numeric_limits<T>::round_style`:

> 185) Equivalent to `FLT_ROUNDS`. Required by ISO/IEC 10967-1:2012.

In C23 5.2.4.2.2 [Characteristics of floating types `<float.h>`], it is specified that a value of `1` for `FLT_ROUNDS` (which equals `round_to_nearest`) means

> to nearest, ties to even

This is also the default ISO/IEC 60559 rounding mode, and chosen by standard libraries such as libstdc++ for `numeric_limits` under that assumption. Do note that C23 no longer references ISO/IEC 10967 in any normative wording, so presumably, matching `FLT_ROUNDS` values means to match the value that exists in C23, including its meaning.

[2026-01-16; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

"IEEE 754-2019 talks about what to do in case of a tie for single digit precision types, such as FP4 (E2M1). Will raise this with WG14." [E2M1 is 1 sign bit, 2 exponent bits, 1 explicit mantissa bit]

Proposed resolution:

This wording is relative to N5014.

1. Modify 17.3.4 [round.style] as indicated:

-1- The rounding mode for floating-point arithmetic is characterized by the values:

(1.1) — `round_indeterminate` if the rounding style is indeterminable
(1.2) — `round_toward_zero` if the rounding style is toward zero
(1.3) — `round_to_nearest` if the rounding style is to the nearest representable value; if there are two equally near such values, the one with an even least significant digit is chosen
(1.4) — `round_toward_infinity` if the rounding style is toward infinity
(1.5) — `round_toward_neg_infinity` if the rounding style is toward negative infinity

---

4477. Placement `operator delete` should be constexpr

Section: 17.6.3.4 [new.delete.placement] **Status:** Tentatively Ready **Submitter:** Jakub Jelinek **Opened:** 2025-11-18 **Last modified:** 2025-11-26

Priority: Not Prioritized

View other active issues in [new.delete.placement].

View all other issues in [new.delete.placement].

Discussion:

The P2747R2 paper made placement operator new constexpr. At that time constexpr exceptions weren't in C++26, so that was all that was needed. But later on when P3068R5 was voted in, the P2747R2 changes look insufficient. The problem is that when you throw from a constructor during operator new, it invokes placement operator delete. And P2747R2 didn't touch that.

This makes it impossible to handle an exception thrown by a placement new-expression during constant evaluation.

[2025-11-26; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 17.6.2 [new.syn] as indicated:

```
constexpr void* operator new  (std::size_t size, void* ptr) noexcept;
constexpr void* operator new[](std::size_t size, void* ptr) noexcept;
constexpr void operator delete  (void* ptr, void*) noexcept;
constexpr void operator delete[](void* ptr, void*) noexcept;
```

2. Modify 17.6.3.4 [new.delete.placement] as indicated:

```
constexpr void* operator new(std::size_t size, void* ptr) noexcept;
```

[…]

```
constexpr void* operator new[](std::size_t size, void* ptr) noexcept;
```

[…]

```
constexpr void operator delete(void* ptr, void*) noexcept;
```

-7- *Effects*: Intentionally performs no action.

-8- *Remarks*: Default function called when any part of the initialization in a placement *new-expression* that invokes the library's non-array placement operator new terminates by throwing an exception (7.6.2.8 [expr.new]).

```
constexpr void operator delete[](void* ptr, void*) noexcept;
```

-9- *Effects*: Intentionally performs no action.

-10- *Remarks*: Default function called when any part of the initialization in a placement *new-expression* that invokes the library's array placement operator new terminates by throwing an exception (7.6.2.8 [expr.new]).

4480. `<stdatomic.h>` should provide `ATOMIC_CHAR8_T_LOCK_FREE`

Section: 32.5.12 [stdatomic.h.syn] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2025-11-21 **Last modified:** 2025-12-04

Priority: Not Prioritized

View all other issues in [stdatomic.h.syn].

Discussion:

Currently, `<stdatomic.h>` is specified to provide `atomic_char8_t` but not its corresponding `ATOMIC_CHAR8_T_LOCK_FREE` macro, which is self-inconsistent. Also, given WG14 N2653 added `ATOMIC_CHAR8_T_LOCK_FREE` to C's `<stdatomic.h>` in C23, perhaps C++ should do the same thing in the spirit of P3348R4.

[2025-12-04; Reflector poll.]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5014.

1. Modify 32.5.12 [stdatomic.h.syn], header `<stdatomic.h>` synopsis, as indicated:

```
template<class T>
  using std-atomic = std::atomic<T>; // exposition only

#define _Atomic(T) std-atomic<T>

#define ATOMIC_BOOL_LOCK_FREE see below
#define ATOMIC_CHAR_LOCK_FREE see below
#define ATOMIC_CHAR8_T_LOCK_FREE see below
#define ATOMIC_CHAR16_T_LOCK_FREE see below
#define ATOMIC_CHAR32_T_LOCK_FREE see below
#define ATOMIC_WCHAR_T_LOCK_FREE see below
#define ATOMIC_SHORT_LOCK_FREE see below
#define ATOMIC_INT_LOCK_FREE see below
#define ATOMIC_LONG_LOCK_FREE see below
#define ATOMIC_LLONG_LOCK_FREE see below
#define ATOMIC_POINTER_LOCK_FREE see below

[…]
```

4481. Disallow `chrono::duration<const T, P>`

Section: 30.5.1 [time.duration.general] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2025-11-26 **Last modified:** 2026-01-16

Priority: Not Prioritized

Discussion:

Using a const type as the `rep` for a `chrono::duration` causes various problems but there seems to be no rule preventing it.

The non-const member operators that modify a `duration` don't work if the `rep` type is const, e.g. `duration<const int>::operator++()` is typically ill-formed (unless the implementation chooses to store `remove_cv_t<rep>` as the data member). `hash<duration<const int>>` uses `hash<const int>` which is not enabled, so you can't hash a `duration` with a const rep. Generic code that wants to perform arithmetic with the `rep` type would need to remember to consistently use `remove_cv_t<rep>` to work correctly with const types.

We should just disallow const `rep` types. If you want a non-modifiable duration, use `const duration<R,P>` not `duration<const R, P>`

[2026-01-16; Reflector poll.]

Set status to Tentatively Ready after ten votes in favour during reflector poll.

This loses support for `duration<volatile T, P>` but that seems OK.

Proposed resolution:

This wording is relative to N5014.

1. Modify 30.5.1 [time.duration.general] as indicated:

   -2- `Rep` shall be an arithmetic type or a class emulating an arithmetic type. If a specialization of `duration` is instantiated with a cv-qualified type or a specialization of `duration` ~~type~~ as the argument for the template parameter `Rep`, the program is ill-formed.

   -3- If `Period` is not a specialization of `ratio`, the program is ill-formed. If `Period::num` is not positive, the program is ill-formed.

   -4- Members of `duration` do not throw exceptions other than those thrown by the indicated operations on their representations.

4483. Multidimensional arrays are not supported by `meta::reflect_constant_array` and related functions

Section: 21.4.3 [meta.define.static] **Status:** Tentatively Ready **Submitter:** Tomasz Kamiński **Opened:** 2025-11-27 **Last modified:** 2026-03-06

Priority: 2

View other active issues in [meta.define.static].

View all other issues in [meta.define.static].

Discussion:

As any array type (even of structural types) is not considered an structural type, per 13.2 [temp.param] p12, any invocation of `reflect_constant_array`/`define_static_array` with a multidimensional array or `span` of arrays is ill-formed due to the *Mandates* in 21.4.3 [meta.define.static] p8 that requires range value type to be structural.

As a consequence, `constant_of` currently supports only single-dimensional arrays (`reflect_constant_array` strips outermost extents), while multi-dimensional arrays are rejected.

Furthermore, `define_static_object` currently uses `define_static_array(span(addressof(t), 1)).data()`, for array types. Since for `T[N]` input this creates an multidimensional `T[1][N]` constant parameter object, this function does not support arrays at all. Creating a distinct template parameter object leads to emission of (otherwise unnecessary) additional symbols, and breaks the invariant that for all supported object types `&constant_of(o) == define_static_object(o)`. We should use `reflect_constant_array` for arrays directly.

The *Throws* clause of `reflect_constant_array` was updated to include any exception thrown by iteration over range.

[2025-12-05; LWG telecon. Wording tweaks]

[2025-12-12; LWG telecon. Further wording tweaks]

[2026-01-16; Reflector poll.]

Set priority to 2 after reflector poll.

"The new *Throws*: paragraph implies that `meta::exception` is thrown if a copy constructor of an array element throws, but we want to propagate the thrown exception not catch it an throw a `meta::exception`. Maybe something like:"

> If evaluation of `reflect_constant` (including evaluation resulting from `reflect_constant_array` invocations) would result in an exception `E` being thrown, then:
>
> - `E` if an exception was thrown by evaluation of an argument (if any), and
> - `meta::exception` otherwise.

"I suggest:"

> *Throws*: Any of
>
> - an exception thrown by any operation on `r` or on iterators and sentinels referring to `r`,
> - an exception thrown by the evaluation of any argument of `reflect_constant` or by any evaluation of `reflect_constant_array`, or
> - `meta::exception` if any invocation of `reflect_constant` would exit via an exception.

Previous resolution [SUPERSEDED]:

This wording is relative to N5014 amended with changes from LWG 4432[(i)].

1. Modify 21.4.3 [meta.define.static] as indicated:

```
template<ranges::input_range R>
  consteval info reflect_constant_array(R&& r);
```

-8- Let ~~T~~U be `ranges::range_value_t<R>` and `T be remove_all_extents_t<U>` ~~e_i be static_cast<T>(*it_i), where it_i is an iterator to the i^{th} element of r~~.

-9- *Mandates*:

(9.1) — `T` is a structural type (13.2 [temp.param]), ~~is_constructible_v<T, ranges::range_reference_t<R>> is true, and~~
(9.2) — `T` satisfies `copy_constructible`, and
(9.3) — if `U` is not an array type, then `is_constructible_v<T, ranges::range_reference_t<R>>` is `true`.

-10- Let `V` be the pack of values of type `info` of the same size as `r`, where the i^{th} element is

(10.1) — `reflect_constant_array(`*it_i*`)` if `U` is an array type,
(10.2) — `reflect_constant(static_cast<T>(`*it_i*`)`~~e_i~~`)` otherwise,

and it_i is an iterator to the i^{th} element of `r`.

-11- Let *P* be

(11.1) — If `sizeof...(V) > 0` is `true`, then the template parameter object (13.2 [temp.param]) of type const `T[sizeof...(V)]` ~~initialized with {[:V:]...}~~, such that `P[`*I*`]` is template-argument-equivalent (13.6 [temp.type]) to the object represented by `V...[`*I*`]` for all *I* in the range [`0`, `sizeof...(V)`).
(11.2) — Otherwise, the template parameter object of type `const array<T, 0>` initialized with `{}`.

-12- *Returns*: `^^`*P*.

-13- *Throws*: Any exception thrown by operations on `r` or on iterators and sentinels referring to `r`. Any exception thrown by the evaluation of any argument of `reflect_constant`~~e_i, or~~ `meta::exception` if ~~evaluation of any `reflect_constant(`e_i`)`~~ any invocation of `reflect_constant` or `reflect_constant_array` would exit via an exception.

[…]

```
template<class T>
  consteval const remove_cvref_t<T>* define_static_object(T&& t);
```

-15- *Effects*:Equivalent to:

```
using U = remove_cvref_t<T>;
if constexpr (meta::is_class_type(^^U)) {
  return addressof(meta::extract<const U&>(meta::reflect_constant(std::forward<T>(t))));
} else if constexpr (meta::is_array_type(^^U)) {
  return addressof(meta::extract<const U&>(meta::reflect_constant_array(std::forward<T>(t))));
} else {
  return define_static_array(span(addressof(t), 1)).data();
}
```

[2026-02-27; LWG telecon. Further wording tweaks]

Rebased on top of N5032

Used alternative suggestion for *Throws* clause.

`define_static_object` now uses `reflect_constant` also for `union` types.

[2026-03-06; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Modify 21.4.3 [meta.define.static] as indicated:

```
template<ranges::input_range R>
  consteval info reflect_constant_array(R&& r);
```

-8- Let ~~`T`~~`U` be `ranges::range_value_t<R>` and `T be remove_all_extents_t<U>` ~~e_i be `static_cast<T>(`$*it_i$`)`, where it_i is an iterator to the i^{th} element of `r`~~.

-9- *Mandates*:

(9.1) — `T` is a structural type (13.2 [temp.param]), ~~`is_constructible_v<T, ranges::range_reference_t<R>>` is `true`, and~~
(9.2) — `T` satisfies `copy_constructible`, and
(9.3) — if `U` is not an array type, then `is_constructible_v<T, ranges::range_reference_t<R>>` is `true`.

-10- Let `V` be the pack of values of type `info` of the same size as `r`, where the i^{th} element is

(10.1) — `reflect_constant_array(`$*it_i$`)` if `U` is an array type,
(10.2) — `reflect_constant(static_cast<T>(`$*it_i$`)`~~e_i~~`)` otherwise,

and it_i is an iterator to the i^{th} element of `r`.

-11- Let *P* be

(11.1) — If `sizeof...(V) > 0` is `true`, then the template parameter object (13.2 [temp.param]) of type const `T[sizeof...(V)]` ~~initialized with `{[:V:]...}`~~, such that `P[`*I*`]` is template-argument-equivalent (13.6 [temp.type]) to the object represented by `V...[`*I*`]` for all *I* in the range `[0, sizeof...(V))`.
(11.2) — Otherwise, the template parameter object of type `const array<T, 0>` initialized with `{}`.

-12- *Returns*: `^^`*P*.

-13- *Throws*: ~~Any exception thrown by the evaluation of any e_i, or `meta::exception` if evaluation of any `reflect_constant(`e_i`)` would exit via an exception.~~ Any of

- -13.1- an exception thrown by any operation on `r` or on iterators and sentinels referring to `r`,
- -13.2- an exception thrown by the evaluation of any argument of `reflect_constant` or by any evaluation of `reflect_constant_array`, or
- -13.3- `meta::exception` if any invocation of `reflect_constant` would exit via an exception.

[…]

```
template<class T>
  consteval const remove_cvref_t<T>* define_static_object(T&& t);
```

-15- *Effects*:Equivalent to:

```
using U = remove_cvref_t<T>;
if constexpr (meta::is_class_type(^^U) || meta::is_union_type(^^U)) {
  return addressof(meta::extract<const U&>(meta::reflect_constant(std::forward<T>(t))));
} else if constexpr (meta::is_array_type(^^U)) {
  return addressof(meta::extract<const U&>(meta::reflect_constant_array(std::forward<T>(t))));
} else {
  return define_static_array(span(addressof(t), 1)).data();
}
```

4491. Rename `submdspan_extents` and `submdspan_canonicalize_slices`

Section: 23.7.3.7 [mdspan.sub] **Status:** Tentatively Ready **Submitter:** Tomasz Kamiński **Opened:** 2025-12-16 **Last modified:** 2026-01-16

Priority: Not Prioritized

Discussion:

Addresses US 152-243 and PL-008.

Rename `submdspan_extents` to `subextents` and `submdspan_canonicalize_slices` to `canonical_slices`.

[2026-01-16; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Modify 23.7.3.2 [mdspan.syn] as follows:

```
// 23.7.3.7 [mdspan.sub], submdspan creation
template<class OffsetType, class LengthType, class StrideType>
  struct strided_slice;

template<class LayoutMapping>
  struct submdspan_mapping_result;

struct full_extent_t { explicit full_extent_t() = default; };
inline constexpr full_extent_t full_extent{};

template<class IndexType, size_t... Extents, class... SliceSpecifiers>
  constexpr auto submdspan_extents(const extents<IndexType, Extents...>&, SliceSpecifiers...);

//  [mdspan.sub.canonical], submdspan slice canonicalization
template<class IndexType, size_t... Extents, class... Slices>
  constexpr auto submdspan_canonicalizecanonical_slices(const extents<IndexType, Extents...>& src,
                                                        Slices... slices);
```

2. Modify [mdspan.sub.canonical] as follows:

`submdspan` slice canonicalization

```
template<class IndexType, size_t... Extents, class... Slices>
  constexpr auto submdspan_canonicalizecanonical_slices(const extents<IndexType, Extents...>& src,
                                                        Slices... slices);
```

3. Modify 23.7.3.7.5 [mdspan.sub.extents] as follows:

23.7.3.7.5 [mdspan.sub.extents] `submdspan_extents` function

```
template<class IndexType, size_t... Extents, class... SliceSpecifiers>
  constexpr auto submdspan_extents(const extents<IndexType, Extents...>& src,
                                   SliceSpecifiers... raw_slices);
```

-1- Let `slices` be the pack introduced by the following declaration:

```
auto [...slices] = submdspan_canonicalizecanonical_slices(src, raw_slices...)
```

4. Modify [mdspan.sub.map.sliceable] as follows:

-5- *Returns*: An object `smr` of type `SMR` such that

- -5.1- `smr.mapping.extents() == submdspan_extents(m.extents(), valid_slices...)` is `true`; and
- -5.2- […]

5. Modify 23.7.3.7.6.1 [mdspan.sub.map.common] as follows:

-5- Let `sub_ext` be the result of `submdspan_extents(extents(), slices...)` and let `SubExtents` be `decltype(sub_ext)`.

6. Modify 23.7.3.7.7 [mdspan.sub.sub] as follows:

-2- Let `slices` be the pack introduced by the following declaration:

```
auto [...slices] = submdspan_canonicalizecanonical_slices(src, raw_slices...)
```

4493. Specification for some functions of bit reference types seems missing

Section: 22.9.2.1 [template.bitset.general], 23.3.14.1 [vector.bool.pspc] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2025-12-16 **Last modified:** 2026-02-18

Priority: Not Prioritized

View all other issues in [template.bitset.general].

Discussion:

We haven't explicitly specified the return values of `bitset<N>::reference::operator bool()` and `vector<bool, A>::reference::operator bool()`, although the intended return values can be inferred from "a class that simulates a reference to a single bit in the sequence". Moreover, specification for `bitset<N>::reference::operator~` seems completely missing, and the comment "flip the bit" seems misleading. Implementations consistently make the `operator~` return `!operator bool()`.

[2026-02-18; Reflector poll.]

Changed resolution to say "the bit referred to by `*this`" instead of "the referred to bit".

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Modify 22.9.2.1 [template.bitset.general] as indicated:

```
[…]
// bit reference
class reference {
public:
  constexpr reference(const reference& x) noexcept;
  constexpr ~reference();
  constexpr reference& operator=(bool x) noexcept; // for b[i] = x;
  constexpr reference& operator=(const reference& x) noexcept; // for b[i] = b[j];
  constexpr const reference& operator=(bool x) const noexcept;
  constexpr operator bool() const noexcept; // for x = b[i];
  constexpr bool operator~() const noexcept; // flips the bit
  constexpr reference& flip() noexcept; // for b[i].flip();

  friend constexpr void swap(reference x, reference y) noexcept;
  friend constexpr void swap(reference x, bool& y) noexcept;
  friend constexpr void swap(bool& x, reference y) noexcept;
};
[…]
```

[…]

```
constexpr reference& reference::operator=(bool x) noexcept;
constexpr reference& reference::operator=(const reference& x) noexcept;
constexpr const reference& reference::operator=(bool x) const noexcept;
```

-6- *Effects*: Sets the bit referred to by `*this` if `bool(x)` is `true`, and clears it otherwise.

-7- *Returns*: `*this`.

`constexpr reference::operator bool() const noexcept;`

-?- *Returns*: `true` if the value of bit referred to by `*this` is one, `false` otherwise.

`constexpr bool reference::operator~() const noexcept;`

-?- *Returns*: `!bool(*this)`.

2. Modify 23.3.14.1 [vector.bool.pspc] as indicated:

[…]

```
constexpr reference& reference::operator=(bool x) noexcept;
constexpr reference& reference::operator=(const reference& x) noexcept;
constexpr const reference& reference::operator=(bool x) const noexcept;
```

-7- *Effects*: Sets the bit referred to by `*this` when `bool(x)` is `true`, and clears it otherwise.

-8- *Returns*: `*this`.

`constexpr reference::operator bool() const noexcept;`

-?- *Returns*: `true` if the value of the bit referred to by `*this` is one, `false` otherwise.

4500. `constant_wrapper` wording problems

Section: 21.3.5 [const.wrap.class] **Status:** Tentatively Ready **Submitter:** Matthias Wippich **Opened:** 2026-01-07 **Last modified:** 2026-01-16

Priority: Not Prioritized

View other active issues in [const.wrap.class].

View all other issues in [const.wrap.class].

Discussion:

During resolution of LWG 4383[(i)] prefix and postfix increment and decrement operators were changed to

```
template<constexpr-param T>
  constexpr auto operator++(this T) noexcept -> constant_wrapper<++Y>
    { return {}; }
template<constexpr-param T>
  constexpr auto operator++(this T, int) noexcept ->
    constant_wrapper<Y++> { return {}; }
template<constexpr-param T>
  constexpr auto operator--(this T) noexcept -> constant_wrapper<--Y>
    { return {}; }
template<constexpr-param T>
  constexpr auto operator--(this T, int) noexcept ->
    constant_wrapper<Y--> { return {}; }
```

However, we do not actually specify what `Y` is. Additionally, the assignment operator has been changed to

```
template<constexpr-param R>
  constexpr auto operator=(R) const noexcept
    -> constant_wrapper<X = R::value> { return {}; }
```

This is grammatically not valid C++. The assignment must be parenthesized.

[2026-01-16; Reflector poll.]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Modify 21.3.5 [const.wrap.class], class template `constant_wrapper` synopsis, as indicated:

```
struct cw-operators { // exposition only
  …
  // pseudo-mutators
  template<constexpr-param T>
    constexpr auto operator++(this T) noexcept
      -> constant_wrapper<~~++Y~~ (++T::value)> { return {}; }
  template<constexpr-param T>
    constexpr auto operator++(this T, int) noexcept
      -> constant_wrapper<~~Y++~~ (T::value++)> { return {}; }
  template<constexpr-param T>
    constexpr auto operator--(this T) noexcept
      -> constant_wrapper<~~--Y~~ (--T::value)> { return {}; }
  template<constexpr-param T>
    constexpr auto operator--(this T, int) noexcept
      -> constant_wrapper<~~Y--~~ (T::value--)> { return {}; }
  …
};

template<cw-fixed-value X, class>
struct constant_wrapper : cw-operators {
  static constexpr const auto & value = X.data;
  using type = constant_wrapper;
  using value_type = decltype(X)::type;

  template<constexpr-param R>
    constexpr auto operator=(R) const noexcept
      -> constant_wrapper<(X = R::value)> { return {}; }
  constexpr operator decltype(auto)() const noexcept { return value; }
};
```

4514. Missing absolute value of `init` in `vector_two_norm` and `matrix_frob_norm`

Section: 29.9.13.9 [linalg.algs.blas1.nrm2], 29.9.13.12 [linalg.algs.blas1.matfrobnorm] **Status:** Tentatively Ready **Submitter:** Mark Hoemmen **Opened:** 2026-01-28 **Last modified:** 2026-02-13

Priority: Not Prioritized

View all other issues in [linalg.algs.blas1.nrm2].

Discussion:

This pull request discussion points out two issues with the current wording of `vector_two_norm`:

1. If `Scalar` is the complex number `1 + i`, then the current wording would make `vector_two_norm` return `sqrt(2i + nonnegative_real_number)`. This would have nonzero imaginary part.

2. If `Scalar` is the real number `-1.0`, then the current wording would make `vector_two_norm` return `sqrt(-1.0 + nonnegative_real_number)`. If `nonnegative_real_number` is less than `1`, then the result would be imaginary.

The analogous issues would also apply to `matrix_frob_norm`.

It's acceptable for the return type `Scalar` to be `complex`. However, the point of letting `init` be nonzero is to support computing the 2-norm of a vector (or the Frobenius norm of a matrix) in multiple steps. The value of `init` should thus always represent a valid 2-norm (or Frobenius norm) result.

There are two ways to fix this:

1. Impose a precondition on `init`, that it have nonnegative real part and zero imaginary part.

2. Change the *Returns* element to take the absolute value of `init`. (The 29.9 [linalg] clause generally uses "absolute value" to mean the magnitude of a complex number, or the absolute value of a non-complex number.)

We offer (2) as the Proposed Fix, as it is consistent with the approach we took for fixing LWG 4136[(i)].

[2026-02-13; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Modify 29.9.13.9 [linalg.algs.blas1.nrm2] as indicated:

> [*Drafting note*: As a drive-by fix the missing closing parenthesis of the `decltype` form has been added.]

```
template<in-vector InVec, scalar Scalar>
  Scalar vector_two_norm(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, scalar Scalar>
  Scalar vector_two_norm(ExecutionPolicy&& exec, InVec v, Scalar init);
```

-1- [*Note 1*: These functions correspond to the BLAS function `xNRM2`[17]. — *end note*]

-2- *Mandates*: `InVec::value_type` and `Scalar` are either a floating-point type, or a specialization of `complex`. Let `a` be *`abs-if-needed`*`(declval<typename InVec::value_type>())` and let `init_abs` be *`abs-if-needed`*`(init)`. Then, `decltype(init_abs * init_abs + a * a)` is convertible to `Scalar`.

-3- *Returns*: The square root of the sum ~~of the square of `init` and the squares of the absolute values of the elements of `v`.~~whose terms are the following:

(3.1) — the square of the absolute value of `init`, and

(3.2) — the squares of the absolute values of the elements of `v`.

[*Note 2*: For `init` equal to zero, this is the Euclidean norm (also called 2-norm) of the vector `v`. — *end note*]

-4- *Remarks*: If `Scalar` has higher precision than `InVec::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

2. Modify 29.9.13.12 [linalg.algs.blas1.matfrobnorm] as indicated:

> [*Drafting note*: As a drive-by fix one spurious `InVec` has been changed to `InMat`]

```
template<in-matrix InMat, scalar Scalar>
  Scalar matrix_frob_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, scalar Scalar>
  Scalar matrix_frob_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
```

-2- *Mandates*: ~~`InVec`~~`InMat``::value_type` and `Scalar` are either a floating-point type, or a specialization of `complex`. Let `a` be *`abs-if-needed`*`(declval<typename InMat::value_type>())` and let `init_abs` be *`abs-if-needed`*`(init)`. Then, `decltype(init_abs * init_abs + a * a)` is convertible to `Scalar`.

-3- *Returns*: The square root of the sum ~~of the squares of `init` and the absolute values of the elements of `A`.~~whose terms are the following:

(3.1) — the square of the absolute value of `init`, and

(3.2) — the squares of the absolute values of the elements of `A`.

[*Note 2*: For `init` equal to zero, this is the Frobenius norm of the matrix `A`. — *end note*]

-4- *Remarks*: If `Scalar` has higher precision than `InMat::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

4517. `data_member_spec` should throw for *cv*-qualified unnamed bit-fields

Section: 21.4.16 [meta.reflection.define.aggregate] **Status:** Tentatively Ready **Submitter:** Marek Polacek **Opened:** 2026-02-05 **Last modified:** 2026-02-27

Priority: Not Prioritized

View all other issues in [meta.reflection.define.aggregate].

Discussion:

DR 2229 changed 11.4.10 [class.bit] p2 to say:

> An unnamed bit-field shall not be declared with a *cv*-qualified type.

It follows that `define_aggregate` should have the same limitation, and thus in:

```
#include <meta>

constexpr auto d1 = std::meta::data_member_spec(^^const int, { .bit_width = 1 });
constexpr auto d2 = std::meta::data_member_spec(^^volatile int, { .bit_width = 1 });
constexpr auto d3 = std::meta::data_member_spec(^^const volatile int, { .bit_width = 1 });
```

all three `data_member_spec` should throw.

[2026-02-27; Reflector poll.]

Set status to Tentatively Ready after 8 votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Modify 21.4.16 [meta.reflection.define.aggregate] as indicated:

```
consteval info data_member_spec(info type, data_member_options options);
```

-4- *Returns*: […]

-5- *Throws*: `meta::exception` unless the following conditions are met:

(5.1) — `dealias(type)` represents either an object type or a reference type;

(5.2) — if `options.name` contains a value, then: […]

(5.3) — if `options.name` does not contain a value, then `options.bit_width` contains a value;

(5.4) — if `options.bit_width` contains a value *V*, then

(5.4.1) — `is_integral_type(type) || is_enum_type(type)` is `true`,

(5.4.2) — `options.alignment` does not contain a value,

(5.4.3) — `options.no_unique_address` is `false`,

(5.4.4) — *V* is not negative, and

(5.4.5) — if *V* equals `0`, then `options.name` does not contain a value; and

(5.4.?) — if `options.name` does not contain a value, then `is_const(type) || is_volatile(type)` is `false`; and

(5.5) — if `options.alignment` contains a value, it is an alignment value (6.8.3 [basic.align]) not less than `alignment_of(type)`.

4522. Clarify that `std::format` transcodes for `std::wformat_strings`

Section: 28.5.2.2 [format.string.std] **Status:** Tentatively Ready **Submitter:** Jan Schultke **Opened:** 2026-02-12 **Last modified:** 2026-02-27

Priority: Not Prioritized

View other active issues in [format.string.std].

View all other issues in [format.string.std].

Discussion:

The current wording in 28.5.2.2 [format.string.std] paragraph 20 suggests that the expression `format(L"{}", 0)` could produce a garbage or malformed string:

> […] Formatting is done as if by calling `to_chars` as specified and copying the output through the output iterator of the format context.

It is not stated that transcoding takes place, so a plausible interpretation is that the output of `to_chars` is copied to the output iterator directly. If the ordinary literal encoding is, say, EBCDIC and the wide literal encoding is, say, UTF-32, simply converting `char` to `wchar_t` is not meaningful.

During the 2026-02-11 SG16 Telecon, when reviewing P3876R0, SG16 examined this issue and concluded that transcoding is intended here, affirmed by the author of P0645R10 Text Formatting. An LWG issue was requested.

[2026-02-27; Reflector poll.]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

P3876 would address that, but we need to clarify the wording for existing standards.

Proposed resolution:

This wording is relative to N5032.

1. Modify 28.5.2.2 [format.string.std] as indicated:

> [*Drafting note*: 28.5.6.5 [format.string.escaped] paragraph 2 similarly references the wide literal encoding (as the associated character encoding for `charT = wchar_t`.]
>
> -20- The meaning of some non-string presentation types is defined in terms of a call to `to_chars`. In such cases, let `[first, last)` be a range large enough to hold the `to_chars` output and value be the formatting argument value. Formatting is done as if by calling `to_chars` as specified , transcoding the `to_chars` output to the wide literal encoding if `charT` is `wchar_t`, and copying the output through the output iterator of the format context.
>
> [*Note 7*: Additional padding and adjustments are performed prior to copying the output through the output iterator as specified by the format specifiers. — *end note*]

4523. `constant_wrapper` should assign to `value`

Section: 21.3.5 [const.wrap.class] **Status:** Tentatively Ready **Submitter:** Tomasz Kamiński **Opened:** 2026-02-17 **Last modified:** 2026-02-27

Priority: Not Prioritized

View other active issues in [const.wrap.class].

View all other issues in [const.wrap.class].

Discussion:

During resolution of LWG 4383(i) assignment operator of `constant_wrapper`, was changed to:

```
template<constexpr-param R>
  constexpr auto operator=(R) const noexcept -> constant_wrapper<X = R::value> { return {}; }
```

This is incorrect (and inconsistent with other operators), as we assign to `X` that is object of specialization of exposition only *`cw-fixed-value`*, instead of `value`.

[2026-02-27; Reflector poll.]

Set status to Tentatively Ready after 9 votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032 with changes from 4500(i).

1. Modify 21.3.5 [const.wrap.class], class template `constant_wrapper` synopsis, as indicated:

```
template<cw-fixed-value X, class>
struct constant_wrapper : cw-operators {
  static constexpr const auto & value = X.data;
  using type = constant_wrapper;
  using value_type = decltype(X)::type;

  template<constexpr-param R>
```

```
    constexpr auto operator=(R) const noexcept
  -> constant_wrapper<(value = R::value)> { return {}; }

  constexpr operator decltype(auto)() const noexcept { return value; }
};
```

4525. `task`'s `final_suspend` should move the result

Section: 33.13.6.5 [task.promise] **Status:** Tentatively Ready **Submitter:** Dietmar Kühl **Opened:** 2026-02-21 **Last modified:** 2026-02-27

Priority: Not Prioritized

View other active issues in [task.promise].

View all other issues in [task.promise].

Discussion:

In 33.13.6.5 [task.promise] p6.3 the `*result` is passed to `set_value` without `std::move`ing: `set_value(std::move(*RCVR*(*this)), *result*)`. Once `set_value` is called the operation state object where *`result`* is stored just gets destroyed. The second argument to `set_value` should be `std::move(*result*)`.

[2026-02-27; Reflector poll.]

Set status to Tentatively Ready after 6 votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Change 33.13.6.5 [task.promise] p6.3 to `std::move` the *`result`*:

```
auto final_suspend() noexcept;
```

-6- *Returns*: An awaitable object of unspecified type (7.6.2.4 [expr.await]) whose member functions arrange for the completion of the asynchronous operation associated with *`STATE`*`(*this)` by invoking:

-6.1- -- `set_error(std::move(`*`RCVR`*`(*this)), std::move(e))` if *`errors`*`.index()` is greater than zero and `e` is the value held by *`errors`*, otherwise

-6.2- -- `set_value(std::move(`*`RCVR`*`(*this)))` if `is_void<T>` is `true`, and otherwise

-6.3- -- `set_value(std::move(`*`RCVR`*`(*this)), std::move(`*`result`*`))`.

4527. `await_transform` needs to use `as_awaitable`

Section: 33.13.6.5 [task.promise] **Status:** Tentatively Ready **Submitter:** Dietmar Kühl **Opened:** 2026-02-21 **Last modified:** 2026-02-27

Priority: Not Prioritized

View other active issues in [task.promise].

View all other issues in [task.promise].

Discussion:

In the `await_transform` overload for `change_coroutine_scheduler` the `return` statement repeats the function name instead of using `as_awaitable`. The `return` statement needs to be fixed. The change may be editorial.

Note that P3941 recommends removing `change_coroutine_scheduler` and the corresponding overload of `await_transform` entirely.

[2026-02-27; Reflector poll.]

Set status to Tentatively Ready after 7 votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Change 33.13.6.5 [task.promise] p10 to use `as_awaitable` in the *Effects* clause:

```
template<class Sch>
  auto await_transform(change_coroutine_scheduler<Sch> sch) noexcept;
```

-10- *Effects*: Equivalent to:

```
return await_transformas_awaitable(just(exchange(SCHED(*this), scheduler_type(sch.scheduler))), *this);
```

4528. `task` needs `get_completion_signatures()`

Section: 33.13.6.2 [task.class] **Status:** Tentatively Ready **Submitter:** Dietmar Kühl **Opened:** 2026-02-21 **Last modified:** 2026-02-27

Priority: Not Prioritized

View other active issues in [task.class].

View all other issues in [task.class].

Discussion:

The changes made by P3164 mean that it isn't sufficient to have a `completion_signatures` type alias in the `task` class. Instead, it needs to have a `static consteval get_completion_signatures()` member function that returns the completion signatures. Instead of defining a type alias, the function can return the same type.

[2026-02-27; Reflector poll.]

Set status to Tentatively Ready after 6 votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Change the `task` synopsis in 33.13.6.2 [task.class] to not have a `completion_signatures` type alias, but instead have a `static consteval get_completion_signatures()` member function that returns the completion signatures:

```
namespace std::execution {
  template<class T = void, class Environment = env<>>
  class task {
    // [task.state]
    template<receiver Rcvr>
      class state;                              // exposition only

  public:
    using sender_concept = sender_t;
    ~~using completion_signatures = see below;~~
    using allocator_type = see below;
    using scheduler_type = see below;
    using stop_source_type = see below;
    using stop_token_type = decltype(declval<stop_source_type>().get_token());
    using error_types = see below;

    // [task.promise]
    class promise_type;

    task(task&&) noexcept;
    ~task();

    template<class Self, class... Env>
      static consteval auto get_completion_signatures();

    template<receiver Rcvr>
      state<Rcvr> connect(Rcvr&& rcvr) &&;

  private:
    coroutine_handle<promise_type> handle;      // exposition only
  };
}
```

2. Remove 33.13.6.2 [task.class] paragraph 4 (the specification moves to 33.13.6.3 [task.members]):

~~4 The type alias `completion_signatures` is a specialization of `execution::completion_signatures` with the template arguments (in unspecified order):~~

~~4.1 — `set_value_t()` if `T` is `void`, and `set_value_t(T)` otherwise;~~

~~4.2 — template arguments of the specialization of `execution::completion_signatures` denoted by `error_types`; and~~

~~4.3 — `set_stopped_t()`.~~

3. Add a new paragraph to 33.13.6.3 [task.members] before the specification of the `connect` member function, with the specification of the new `get_completion_signatures` member function:

```
template<class Self, class... Env>
  static consteval auto get_completion_signatures();
```

-?- Let the type `C` be a specialization of `execution::completion_signatures` with the template arguments (in unspecified order):

-?.1- — `set_value_t()` if `T` is `void`, and `set_value_t(T)` otherwise;

-?.2- — template arguments of the specialization of `execution::completion_signatures` denoted by `error_types`; and

-?.3- — `set_stopped_t()`.

-?- *Returns*: `C()`.

4529. `task::promise_type::await_transform` declaration and definition mismatch

Section: 33.13.6.5 [task.promise] **Status:** Tentatively Ready **Submitter:** Dietmar Kühl **Opened:** 2026-02-21 **Last modified:** 2026-02-27

Priority: Not Prioritized

View other active issues in [task.promise].

View all other issues in [task.promise].

Discussion:

The declarations of `await_transform` mismatch between the synopsis in 33.13.6.5 [task.promise] and the definition in 33.13.6.5 [task.promise] paragraph 9:

```
template<class A>
  auto await_transform(A&& a);
template<class Sch>
  auto await_transform(change_coroutine_scheduler<Sch> sch);
```

vs.

```
template<sender Sender>
  auto await_transform(Sender&& sndr) noexcept;
template<class Sch>
  auto await_transform(change_coroutine_scheduler<Sch> sch) noexcept;
```

The declaration should be fixed to match the definition.

[2026-02-27; Reflector poll.]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5032.

1. Change the declarations of `await_transform` in the synopsis of `promise_type` in 33.13.6.5 [task.promise] to match the definition in 33.13.6.5 [task.promise] p9 except for omitting the `noexcept` specifier:

```
namespace std::execution {
  template<class T, class Environment>
  class task<T, Environment>::promise_type {
  public:
    ...
    template<class E>
      unspecified yield_value(with_error<E> error);

    template<~~class A~~sender Sender>
      auto await_transform(~~A&&~~ aSender&& sndr);
    template<class Sch>
      auto await_transform(change_coroutine_scheduler<Sch> sch);

    unspecified get_env() const noexcept;
    ...
  };
}
```

2. Remove the `noexcept` specifier from the declarations of `await_transform` in the definition of `promise_type` in 33.13.6.5 [task.promise] p9:

```
template<sender Sender>
  auto await_transform(Sender&& sndr) ~~noexcept~~;
```

-9- *Returns*: If `same_as<inline_scheduler, scheduler_type>` is `true` returns `as_awaitable(std::forward<Sender>(sndr), *this);` otherwise returns `as_awaitable(affine_on(std::forward<Sender>(sndr), SCHED(*this)), *this)`.

```
template<class Sch>
  auto await_transform(change_coroutine_scheduler<Sch> sch) ~~noexcept~~;
```

-10- *Effects*: Equivalent to:

```
    return await_transform(just(exchange(SCHED(*this), scheduler_type(sch.scheduler))), *this);
```